



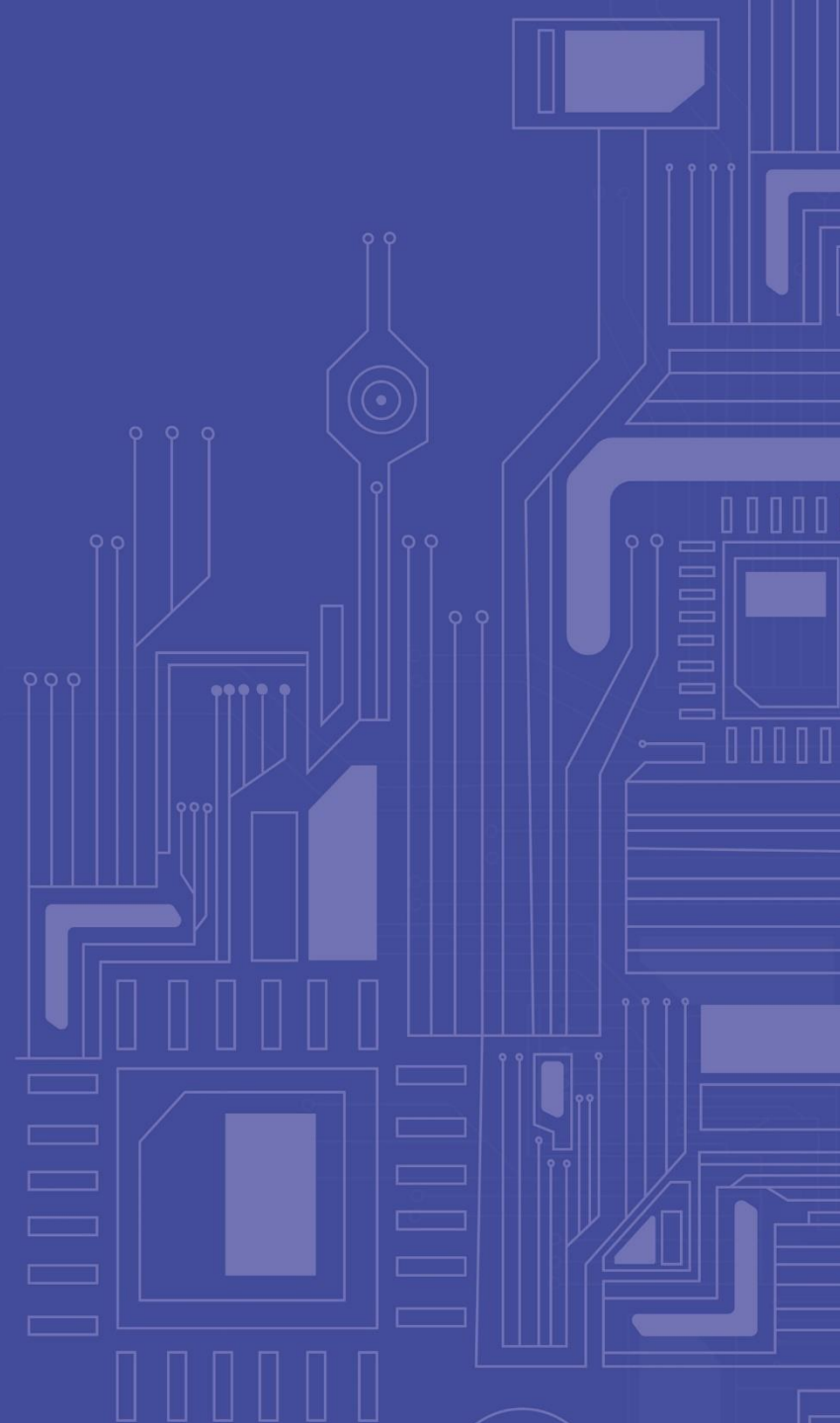
МИНОБРНАУКИ
РОССИИ



Передовые
инженерные
школы

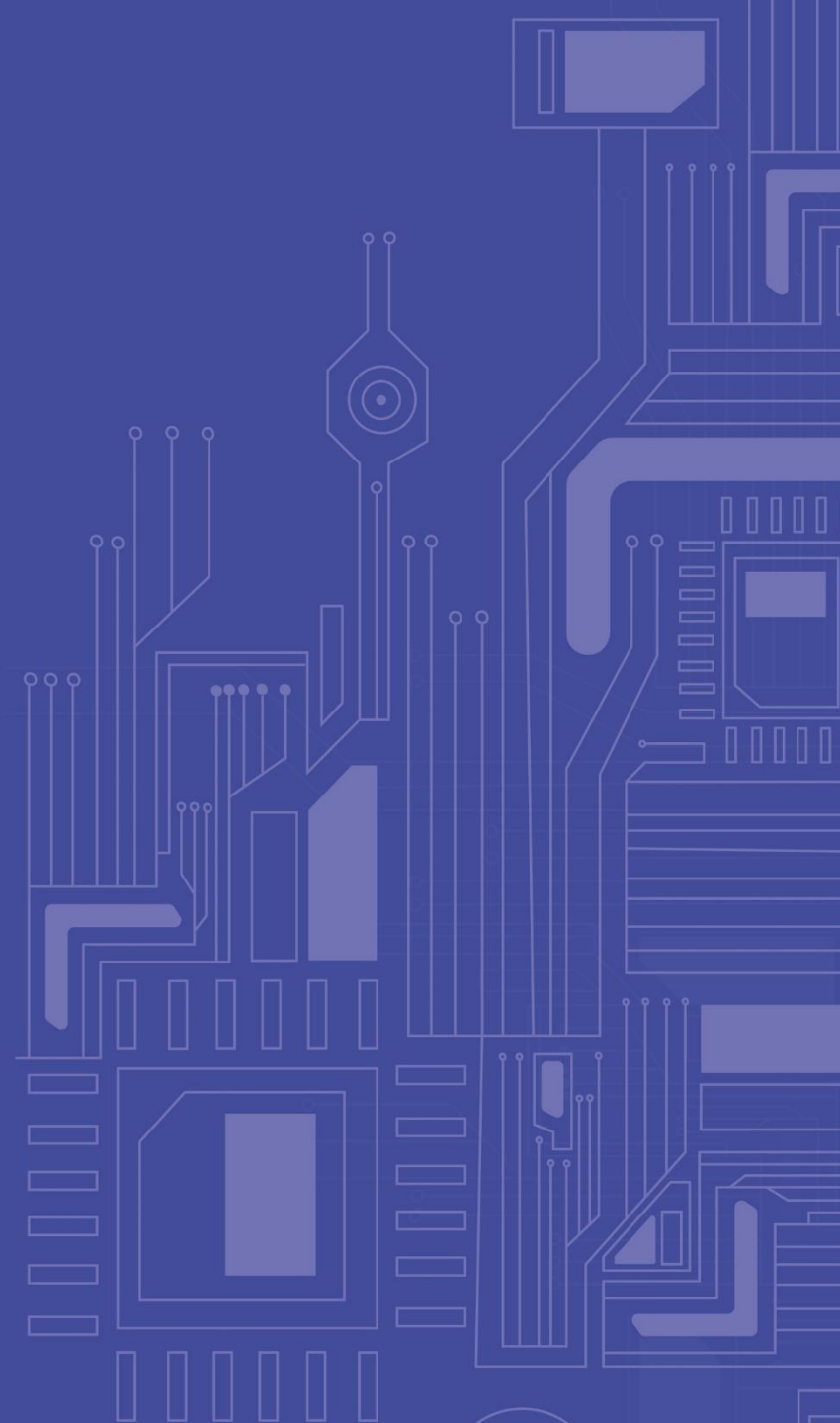
СИСТЕМЫ УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ

Лекция 8



- Сериализация
 - Журнализация
 - Восстановление после сбоев
 - Введение в SQL

СЕРИАЛИЗАЦИЯ НА ОСНОВЕ ВЕРСИОННОСТИ



Основная идея этих алгоритмов сериализации транзакций состоит в том, что в базе данных **допускается существование нескольких «версий» одного и того же объекта**. Эти алгоритмы, главным образом, направлены на преодоление конфликтов транзакций категорий R/W и W/R, позволяя выполнять операции чтения над некоторой предыдущей версией объекта базы данных. В результате операции чтения выполняются без задержек и тупиков, а также без некоторых откатов, возможных при применении метода временных меток. *Поддерживается Oracle и PostgreSQL*

Версионный вариант алгоритма временных меток *(Multiversion Timestamp Ordering, MVTO)*

➡ Как и в простом методе временных меток, в алгоритме MVTO порядок выполнения операций одновременно выполняемых транзакций задается порядком временных меток, которые получают транзакции во время старта. Временные метки также используются для идентификации версий данных при чтении и модификации – каждая версия получает временную метку той транзакции, которая ее записала.

➡ Алгоритм MVTO не только следит за порядком выполнения операций транзакций, но также отвечает за трансформацию операций над объектами базы данных в операции над версиями этих объектов, т.е. каждая операция над объектом базы данных *o* преобразуется в соответствующую операцию над некоторой версией объекта *o*.

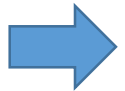
При описании алгоритма будем использовать следующие обозначения.

- Временную метку, полученную транзакцией T_i в начале ее работы, будем обозначать как $t(T_i)$.
- Операция чтения объекта базы данных o , выполняемая в транзакции T_i , будет обозначаться как $R_i(o)$.
- Для обозначения того, что транзакция T_i читает версию объекта базы данных o , созданную транзакцией T_k , будем использовать запись $R_i(o_k)$.
- Для обозначения того, что транзакция T_i записывает версию элемента данных o , будем использовать запись $W_i(o_i)$.

Алгоритм MVTO работает следующим образом.

1. Любая операция $R_i(o)$ преобразуется в операцию $R_i(o_k)$, где o_k – это версия объекта o , помеченная наибольшей временной меткой $t(T_k)$, такой что $t(T_k) \leq t(T_i)$. Другими словами, транзакции T_i для чтения дается версия объекта o , созданная транзакцией T_k , которая не моложе T_i , но старше любой другой транзакции T_n , создававшей свою версию объекта o .
2. При обработке операции $W_i(o)$ выполняются следующие действия:
 - если к этому времени некоторой незафиксированной транзакцией T_n уже выполнена некоторая операция $R_n(o_k)$, такая что $t(T_k) \leq t(T_i) < t(T_n)$, то операция $W_i(o)$ не выполняется, а транзакция T_i откатывается;
 - в противном случае $W_i(o)$ преобразуется в $W_i(o_i)$, т.е. образуется еще одна версия объекта o .

3. При откате любой транзакции уничтожаются все созданные ею версии объектов базы данных и откатываются все транзакции, прочитавшие хотя бы одну из этих версий. Тем самым, откаты транзакций могут быть «каскадными».
4. Выполнение операции фиксации транзакции T_i (COMMIT) откладывается до того момента, когда завершатся все транзакции, записавшие версии данных, прочитанные T_i . Легко видеть, что без соблюдения этого требования не соблюдалось бы свойство долговечности (durability) транзакций, поскольку при откате некоторых транзакций потребовалось бы откатывать и ранее зафиксированные транзакции.



Основным преимуществом алгоритма MVTO является отсутствие задержек и откатов при выполнении операций чтения, а основным недостатком – возможность возникновения каскадных откатов транзакций при выполнении операций записи. Кроме того, в базе данных может накапливаться произвольное число версий одного и того же объекта, и определение того, какие версии больше не требуются, является серьезной технической проблемой.

ПРИМЕР АЛГОРИТМА MVTO



1

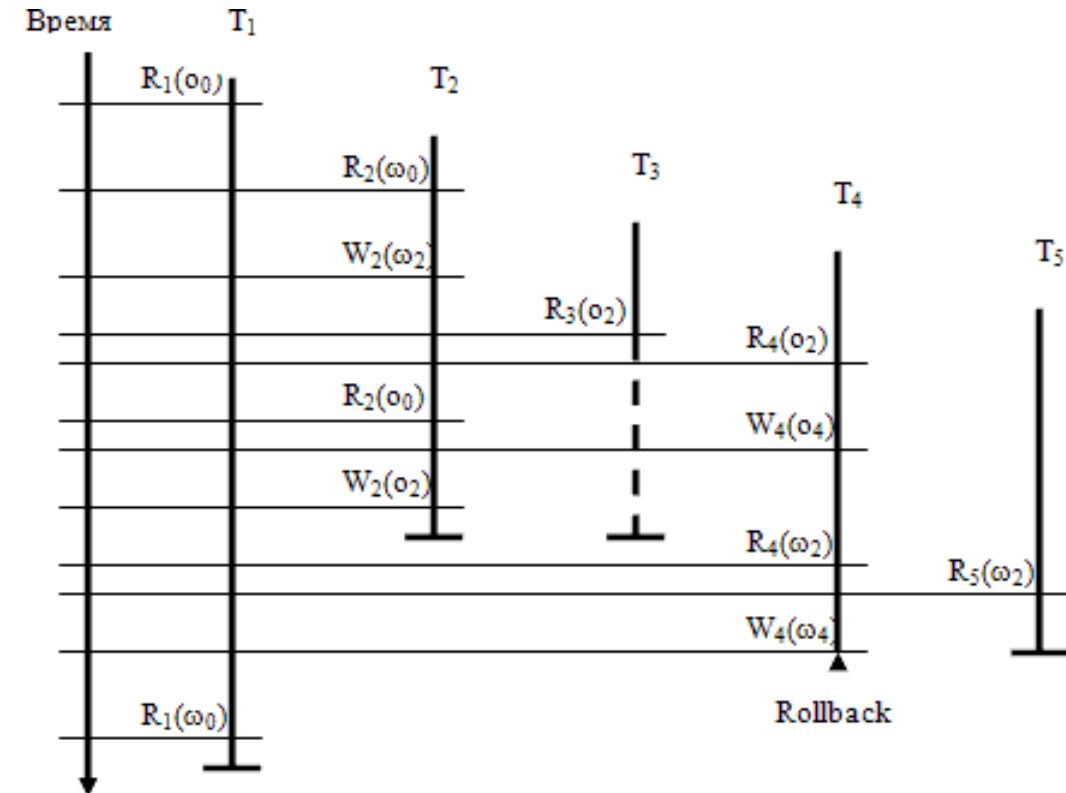
При использовании метода блокировок между T_1 и T_2 возник бы синхронизационный тупик, а при использовании обычного метода временных меток одна из транзакций подверглась бы откату. Однако при применении версий такие неприятности не возникают из-за того, что первая транзакция читает «старые» версии объектов o и ω .

2

Транзакция T_3 ожидает фиксации транзакции T_2 перед своим собственным завершением (на рис. это показано пунктирной линией). Это происходит потому, что транзакция T_3 прочитала версию o_2 объекта o , образованную еще не зафиксированной транзакцией.

3

Транзакция T_4 пытается создать версию ω_4 объекта ω после того, как еще не зафиксированная транзакция T_5 (начавшаяся позже) уже прочитала более раннюю версию ω_4 . Поэтому транзакция T_5 не сможет «увидеть» изменения объекта ω , произведенные транзакцией T_4 . Следовательно, сериализация транзакций в порядке получения ими временных меток становится невозможной, и приходится произвести откат транзакции T_4 .



Версионный вариант двухфазного протокола синхронизационных блокировок

При описании двухверсионного варианта протокола 2PL (***Two-Version Two-Phase Locking Protocol, 2V2PL***) будем называть *текущими* версиями объектов базы данных версии, созданные зафиксированными транзакциями с наиболее поздним временем фиксации; *незафиксированными* версиями – версии, созданные еще незавершившимися транзакциями.

При следовании протоколу 2V2PL в каждый момент времени существует не более одной незафиксированной версии каждого объекта базы данных.

Операции любой транзакции T_i над объектом базы данных o обрабатываются следующим образом:

- операция $R_i(o)$ немедленно выполняется над текущей версией объекта o ;
- операция $W_i(o)$, приводящая к созданию новой версии объекта o , выполняется только после завершения (фиксации или отката) транзакции, создавшей незафиксированную версию объекта o ;
- выполнение операции COMMIT откладывается до тех пор, пока не завершатся все транзакции T_k , прочитавшие текущие версии объектов базы данных, которые должны замениться незафиксированными версиями этих объектов, созданными транзакцией T_i .

Для реализации 2V2PL используются три типа блокировок:

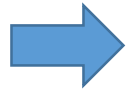
- **RL (Read Lock)** – в этом режиме блокируется любой объект базы данных *o* перед выполнением операции чтения его текущей версии; удержание этой блокировки до конца транзакции гарантирует, что при повторном чтении объекта *o* будет прочитана та же версия этого объекта;
- **WL (Write Lock)** – в этом режиме блокируется любой объект базы данных *o* перед выполнением операции, приводящей к созданию новой (незафиксированной) версии этого объекта; удержание этой блокировки до конца транзакции гарантирует, что в любой момент времени будет существовать не более одной незафиксированной версии любого объекта базы данных;
- **CL (Commit Lock)** – блокировка устанавливается во время выполнения операции COMMIT транзакции и затрагивает любой объект базы данных, новую версию которого создала данная транзакция; удовлетворение этой блокировки для данной транзакции гарантирует, что завершились все транзакции, читавшие текущие версии объектов, новые версии которых были созданы при выполнении данной транзакции, и, следовательно, их можно заменить.

Совместимость блокировок

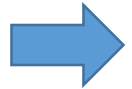
	RL(o)	WL(o)	CL(o)
RL(o)	да	да	нет
WL(o)	да	нет	нет
CL(o)	нет	нет	нет

- Операция чтения может блокироваться только на время фиксации транзакции, заменяющей текущую версию требуемого объекта базы данных.
- Для выполнения операции записи требуется долговременная монополярная блокировка соответствующего объекта базы данных, которая, совместима с блокировкой по чтению.
- Возможны синхронизационные тупики.

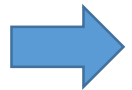
Версионно-блокировочный протокол сериализации транзакций для поддержки только читающих транзакций (*Multiversion Protocol for Read-Only Transactions, ROMV*)



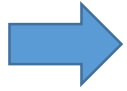
При применении этого протокола при образовании **каждой транзакции явно указывается ее тип** – только читающая (read-only) или изменяющая (update) транзакция. В только читающих транзакциях допускается использование только операций чтения объектов базы данных, а в изменяющих транзакциях – операций и чтения, и записи.



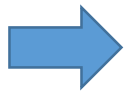
Изменяющие транзакции выполняются в соответствии с обычным протоколом 2PL, т.е. перед выполнением операции чтения или записи объекта базы данных о этот **объект должен быть заблокирован в режиме S или X соответственно**, и блокировки объектов удерживаются до конца изменяющей транзакции. Каждая операции записи объекта о создает его новую версию, которая при завершении транзакции помечается временной меткой, соответствующей моменту фиксации этой транзакции.



Каждая **только читающая транзакция** при своем образовании получает соответствующую временную метку. При выполнении операции чтения объекта базы данных о транзакция получает доступ к версии объекта о, образованной изменяющей транзакцией, которая хронологически последней зафиксировалась к моменту образования данной читающей транзакции.

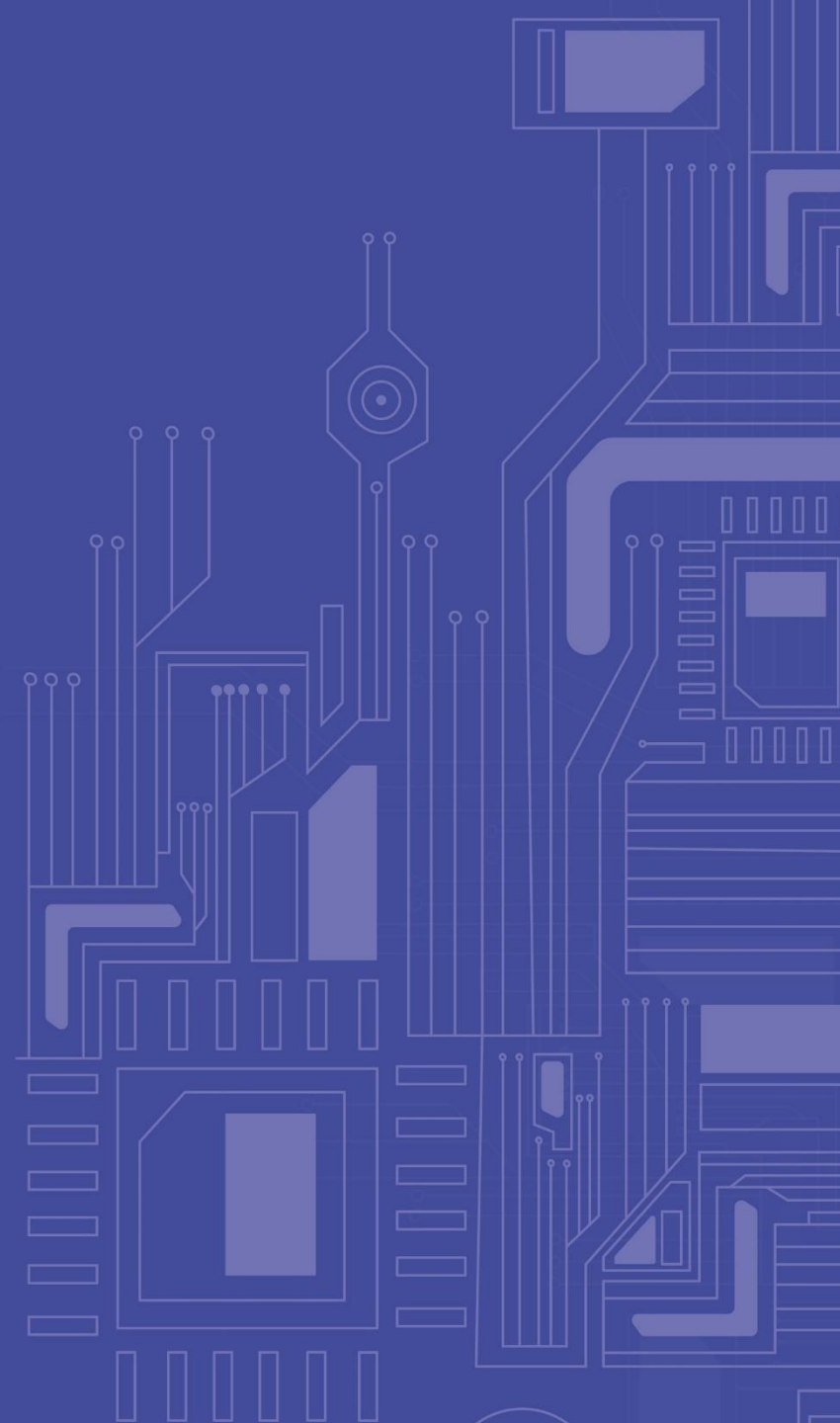


Основным плюсом протокола ROMV по сравнению с ранее описанным протоколом 2V2PL является принципиальное отсутствие синхронизационных задержек при выполнении операций чтения только читающих транзакций. Если сравнивать ROMV с MVTO, то он выигрывает в принципиальном отсутствии откатов только читающих транзакций. Конечно, при работе изменяющих транзакций возможно возникновение синхронизационных тупиков и откатов, и здесь требуется использовать обычные методы распознавания и разрушения тупиков.



Кроме того, при использовании протокола ROMV в базе данных может возникать произвольное число версий объектов. Требуется создание специального сборщика мусора, который должен удалять ненужные версии данных. Простейший сборщик мусора удаляет все неиспользуемые версии, значения временных меток которых меньше значения временной метки старейшей активной только читающей транзакции.

ЖУРНАЛИЗАЦИЯ (ЛОГИ)



Общей целью журнализации изменений баз данных является обеспечение возможности восстановления согласованного состояния базы данных после любого сбоя.

Общими принципами восстановления являются следующие:

- результаты зафиксированных транзакций должны быть сохранены в восстановленном состоянии базы данных (т.е. должно поддерживаться свойство *долговечности (durability)* транзакций);
- результаты незафиксированных транзакций должны отсутствовать в восстановленном состоянии базы данных (в противном случае состояние базы данных могло бы оказаться не целостным).

Возможны следующие ситуации, при которых требуется производить восстановление состояния базы данных:

- Индивидуальный откат транзакции. Тривиальной ситуацией отката транзакции является ее явное завершение оператором ROLLBACK.
- Восстановление после внезапной потери содержимого оперативной памяти (мягкий сбой). Такая ситуация может возникнуть при аварийном выключении электрического питания, при возникновении неустранимого сбоя процессора (например, срабатывании контроля основной памяти) и т.д. Ситуация характеризуется потерей буфера оперативной памяти СУБД.
- Восстановление после поломки основного внешнего носителя базы данных (жесткий сбой).

Во всех трех случаях основой восстановления является хранение избыточных данных. Эти избыточные данные хранятся в журнале, содержащем последовательность записей об изменении базы данных.

Рассматриваем подход общего журнала СУБД, без локальных журналов транзакций (что тоже возможно, но более затратно). Рассматриваем старые классические простые принципы работы СУБД (System R).

➔ По причинам объективно существующей разницы в скорости работы процессоров и основной памяти и устройств внешней памяти (эта разница в скорости существовала, существует, и будет существовать всегда) **буферизация блоков базы данных** в основной памяти является единственным реальным способом достижения приемлемой эффективности СУБД. Без поддержки буферизации базы данных СУБД работала бы со скоростью магнитных дисков, т.е. на несколько порядков медленнее, чем если бы обработка данных происходила в основной памяти.

➔ Поэтому записи в журнал тоже буферизуются: при нормальной работе буфер выталкивается во внешнюю память журнала только при полном заполнении записями. Более точно, для буферизации записей журнала обычно используются два буфера.

- После полного заполнения **первый буфер** выталкивается на магнитный диск, и пока совершается этот обмен, журнальные записи размещаются во втором буфере.
- К моменту конца обмена заполняется **второй буфер**, он выталкивается во внешнюю память, а журнальные записи снова размещаются в первом буфере и т.д.

Используются буфера (и базы данных, и журнала), располагающиеся именно в физической основной памяти, управляемой непосредственно СУБД, а не виртуальной памяти СУБД, управляемой операционной системой. Использование буферов виртуальной памяти является практически бессмысленным делом, поскольку в этом случае операционная система, руководствуясь своими собственными стратегиями управления основной памяти, в любой момент может удалить буферную страницу СУБД из основной памяти и перенести ее копию во внешнюю память.

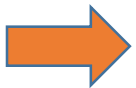
ОС

Если некоторый процесс требует обеспечения доступа к странице виртуальной памяти, отсутствующей в основной памяти, и нет свободных страниц основной памяти, в соответствии с некоторым критерием выбирается некоторая занятая страница основной памяти, освобождается (т.е. изымается из виртуальной памяти какого-то процесса и, может быть, копируется на диск) и подключается к виртуальной памяти запросившего процесса с предварительным считыванием с диска нужных данных.

СУБД

Если при выполнении некоторой операции в некоторой транзакции требуется доступ к некоторому блоку базы данных, и копия этого блока отсутствует в буферном пуле, СУБД должна выделить какую-либо страницу буферного пула, считать в нее с диска требуемый блок базы данных и предоставить доступ к этой странице запросившей операции. Конечно, в буферном пуле может не оказаться свободных страниц, и тогда СУБД в соответствии с некоторым критерием находит некоторую занятую страницу, освобождает ее (возможно, выталкивает во внешнюю память).

Почти всегда ОС стремится заменить страницу, к которой предположительно дольше всего не будет обращений, но, поскольку предвидение будущего невозможно, оно аппроксимируется прошлым.



В частности, в одном из популярных алгоритмов замещения страниц **LRU (Least Recently Used)** принимается предположение, что дольше всего в будущем не потребуется та страница, к которой дольше всего не обращались в прошлом.

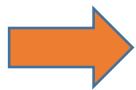
МЕТОД LRU ДЛЯ СУБД



В стратегии замещения страниц буферного пула СУБД тоже чаще всего используется некоторая разновидность алгоритма LRU. Но, как уже отмечалось выше, СУБД располагает большей информацией о страницах буферного пула, чем операционная система о страницах основной памяти.

ПРИМЕР Если в некоторой транзакции выполняется сканирование некоторой таблицы без использования индекса, и при выполнении операции NEXT был затребован доступ к некоторому блоку базы данных (с соответствующим перемещением копии этого блока в некоторую страницу буферного пула), то подсистема управления буферным пулом «знает», что эта страница еще точно потребуется до тех пор, пока не будет прочитан последний кортеж сканируемой таблицы, располагающийся в данной странице. Более того, СУБД «знает», какой блок базы данных потребуется после завершения просмотра кортежей данного блока, и может заранее переместить его копию в некоторую страницу буферного пула.

КОРНЕВЫЕ БЛОКИ При любом просмотре таблицы на основе некоторого индекса гарантированно потребуется доступ к корневому блоку соответствующего В-дерева. При вставке кортежа в любую таблицу или удалении из нее кортежа будет необходимо должным образом изменить все определенные для нее индексы, и для этого тоже гарантированно потребуется доступ к корневым блокам всех соответствующих В-деревьев.



В стратегии замещения страниц буферного пула базы данных обычно используется алгоритм LRU с приоритетами страниц. В частности, страницы, содержащие копии корневых блоков индексов, являются настолько высокоприоритетными, что обычно никогда не замещаются. Кроме того, поддерживается предварительное считывание в буферную память копий блоков, доступ к которым вскоре понадобится.

ФИЗИЧЕСКАЯ СИНХРОНИЗАЦИЯ (1/2)



Поскольку в СУБД может одновременно («параллельно») выполняться несколько транзакций, вполне реальна ситуация, когда в двух одновременно выполняемых операциях требуется доступ к одному и тому же блоку базы данных (т.е. к одной и той же буферной странице, содержащей копию этого блока). Понятно, что в одновременном доступе для чтения содержимого блока ничего плохого нет, но параллельное изменение блока может привести к непредсказуемым результатам.

ПРИМЕР 1

Предположим, что в двух параллельно выполняемых транзакциях одновременно выполняются операции модификации кортежей, у одного из которых $tid = (n, 1)$, а у другого $tid = (n, 2)$. Если в СУБД используются блокировки на уровне кортежей, то система допустит параллельное выполнение этих двух операций, и они будут одновременно изменять страницу, содержащую копию блока базы данных с номером n . При выполнении обеих операций может потребоваться перемещение кортежей внутри этого блока, и понятно, что в результате ничего хорошего, скорее всего, не получится. Также, логическая синхронизация может легко допустить параллельное выполнение нескольких операций, требующих обновления одного и того же индекса. Некоординированное параллельное обновление В-дерева с большой вероятностью приводит к разрушению его структуры.

Поэтому **при выполнении операций уровня RSS необходимо поддерживать дополнительную «физическую» синхронизацию**, в которой единицами блокировки служат страницы буферного пула (или блоки) базы данных. В пределах операции перед чтением из страницы буферного пула (блока базы данных) требуется запросить у подсистемы управления буферным пулом блокировку соответствующей страницы (блока) в режиме S, а перед записью в страницу (в блок) – ее блокировку в режиме X.

ПРИМЕР 2

При выполнении операций уровня RSS могут возникать ошибки, обнаруживаемые в середине операции, уже после того, как одна или несколько страниц буферного пула (блоков базы данных) было изменено. Например, может выполняться операция вставки кортежа в некоторую таблицу, нарушающая уникальность некоторого индекса, определенного над этой таблицей. Нарушение уникальности этого индекса будет обнаружено при попытке вставить в него новый ключ, но до этого новый кортеж уже мог быть размещен в блоке данных, и некоторые индексы уже могли быть успешно обновлены.

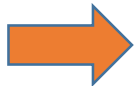
При обнаружении ошибки операции нужно ликвидировать все ее следы в базе данных и выдать соответствующий код ошибки на уровень RDS. Проще всего сделать это, произведя обратные изменения всех страниц (блоков базы данных), которые были изменены при прямом выполнении операции. Но для этого **требуется, чтобы все страницы (блоки базы данных), заблокированные при выполнении операции, оставались заблокированными до конца этой операции.**

Тем самым, для подсистемы управления буферным пулом операции уровня RSS являются (почти) тем же, чем являются транзакции для подсистемы управления транзакциями. **Достаточным условием корректного выполнения операций является соблюдение двухфазного протокола синхронизационных блокировок над страницами буферного пула в пределах операций.**

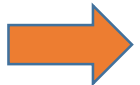
Имеются два вида буферов – буфера журнала и буферный пул страниц основной памяти, – которые содержат связанную информацию. И те, и другие буфера могут выталкиваться во внешнюю память.

- Основной причиной выталкивания буфера журнала является его полное заполнение журнальными записями.
- Страницы буферного пула базы данных чаще всего выталкиваются во внешнюю память, когда требуется переместить в основную память некоторый блок базы данных, а свободных страниц в буферном пуле нет. Тогда срабатывает алгоритм замещения страниц, выбирается страница, содержимое которой, вероятно, дольше всего не потребуется, и эта страница (если ее содержимое изменялось) выталкивается в соответствующий блок внешней памяти базы данных.

Проблема состоит в выработке некоторой общей политики выталкивания, которая обеспечивала бы возможность восстановления состояния базы данных после сбоев.

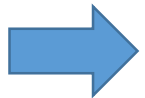


Эта проблема не возникает при индивидуальных откатах транзакций, поскольку в этих случаях содержимое основной памяти не утрачено, и при восстановлении можно пользоваться содержимым как буфера журнала, так и буферных страниц базы данных.



Но если произошел мягкий сбой, и содержимое буферов утрачено, то для проведения восстановления базы данных необходимо иметь некоторое согласованное состояние журнала и базы данных во внешней памяти.

Основным принципом согласованной политики выталкивания буфера журнала и буферных страниц базы данных является то, что запись об изменении объекта базы данных должна оказаться во внешней памяти журнала раньше, чем измененный объект окажется во внешней памяти базы данных. Соответствующий протокол журнализации (и управления буферизацией) называется **WAL (Write Ahead Log, «пиши сначала в журнал»)** и состоит в том, что если требуется вытолкнуть во внешнюю память буферную страницу, содержащую измененный объект базы данных, то перед этим нужно гарантировать выталкивание во внешнюю память журнала буферной страницы журнала, содержащей запись об изменении этого объекта.



Дополнительное условие на выталкивание буферов накладывается тем требованием, что **каждая успешно завершенная транзакция должна быть реально зафиксирована во внешней памяти.** Какой бы сбой не произошел, система должна быть в состоянии восстановить состояние базы данных, содержащее результаты всех транзакций, зафиксированных до момента сбоя.



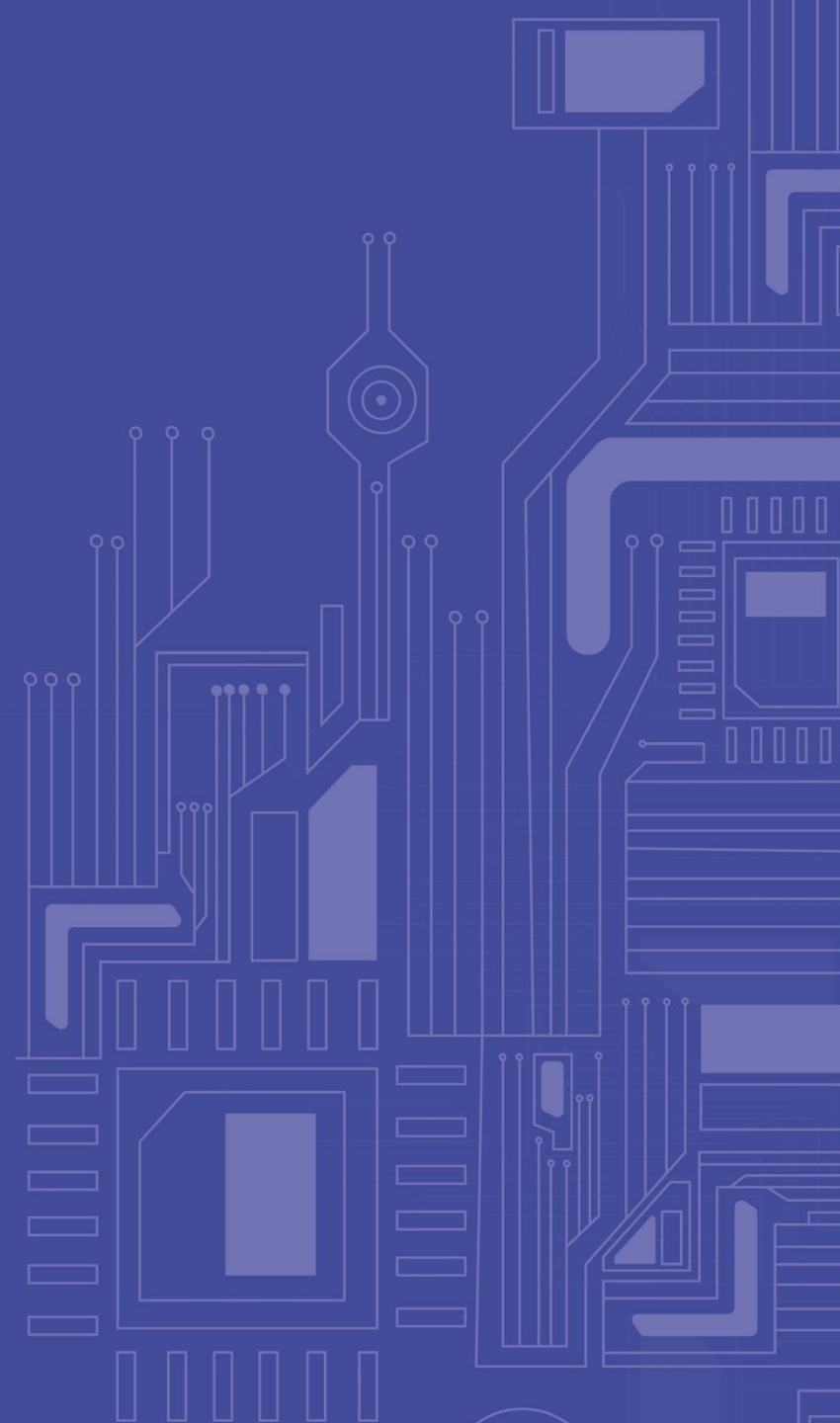
Минимальным требованием, гарантирующим возможность восстановления последнего согласованного состояния базы данных, является **выталкивание при фиксации транзакции во внешнюю память журнала всех записей об изменении базы данных этой транзакцией.** При этом последней записью в журнал, производимой от имени данной транзакции, является специальная запись о конце транзакции.

- ▶ Для обеспечения возможности индивидуального отката транзакции по общему журналу все записи в журнале от данной транзакции связываются в **обратный список**.
- ▶ **В начале списка** для незавершенных транзакций находится запись о последнем изменении базы данных, произведенном данной транзакцией. Заметим, что в этом случае хронологически последние записи могут быть еще не вытолкнуты во внешнюю память журнала и могут находиться в буфере основной памяти.
- ▶ **Для закончившихся транзакций** (индивидуальные откаты которых уже невозможны) началом списка является запись о конце транзакции, которая обязательно вытолкнута во внешнюю память журнала, т.е. весь список находится во внешней памяти.
- ▶ **Концом списка** всегда служит первая запись об изменении базы данных, произведенном данной транзакцией.
- ▶ Обычно в каждой записи **проставляется уникальный идентификатор транзакции**, чтобы можно было восстановить прямой список записей об изменениях базы данных данной транзакцией.

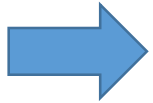
Индивидуальный откат транзакции (это возможно только для незавершенных транзакций) выполняется следующим образом:

- Выбирается очередная журнальная запись из списка данной транзакции.
- Выполняется противоположная по смыслу операция: вместо операции INSERT выполняется соответствующая операция DELETE, вместо операции DELETE выполняется INSERT, и вместо прямой операции UPDATE – обратная операция UPDATE, восстанавливающая предыдущее состояние объекта базы данных.
- Любая из этих обратных операций также журнализуется. Собственно для индивидуального отката это не нужно, но при выполнении индивидуального отката транзакции может произойти мягкий сбой, при восстановлении после которого потребуются откатить транзакции, для которых не полностью выполнен индивидуальный откат.
- При успешном завершении отката в журнал заносится запись о конце транзакции. С точки зрения журнала такая транзакция является зафиксированной.

ВОССТАНОВЛЕНИЕ ПОСЛЕ СБОЕВ



Мягкий сбой характеризуется утратой оперативной памяти системы. При этом поражаются все выполняющиеся в момент **сбоя** транзакции, теряется содержимое всех буферов базы данных. Данные, хранящиеся на диске, остаются неповрежденными.



После мягкого сбоя набор блоков внешней памяти базы данных может оказаться несогласованным, т.е. часть блоков внешней памяти соответствует объекту до изменения, часть – после изменения. Например, в результате выполнения операции UPDATE соответствующий кортеж мог переместиться в другой блок. В этом случае изменяются два блока: в описатель кортежа в его исходном блоке записывается его новый tid, а в новом блоке размещается сам модифицированный кортеж. Очевидно, что если хотя бы один из этих блоков не попал во внешнюю память базы данных к моменту мягкого сбоя, то при восстановлении не удастся вернуть кортеж на его прежнее место. Другими словами, к такому состоянию внешней памяти базы данных не применимы операции логического уровня.

Состояние внешней памяти базы данных называется **физически согласованным**, если наборы страниц всех объектов согласованы, т.е. соответствуют состоянию любого объекта либо после его изменения, либо до изменения.

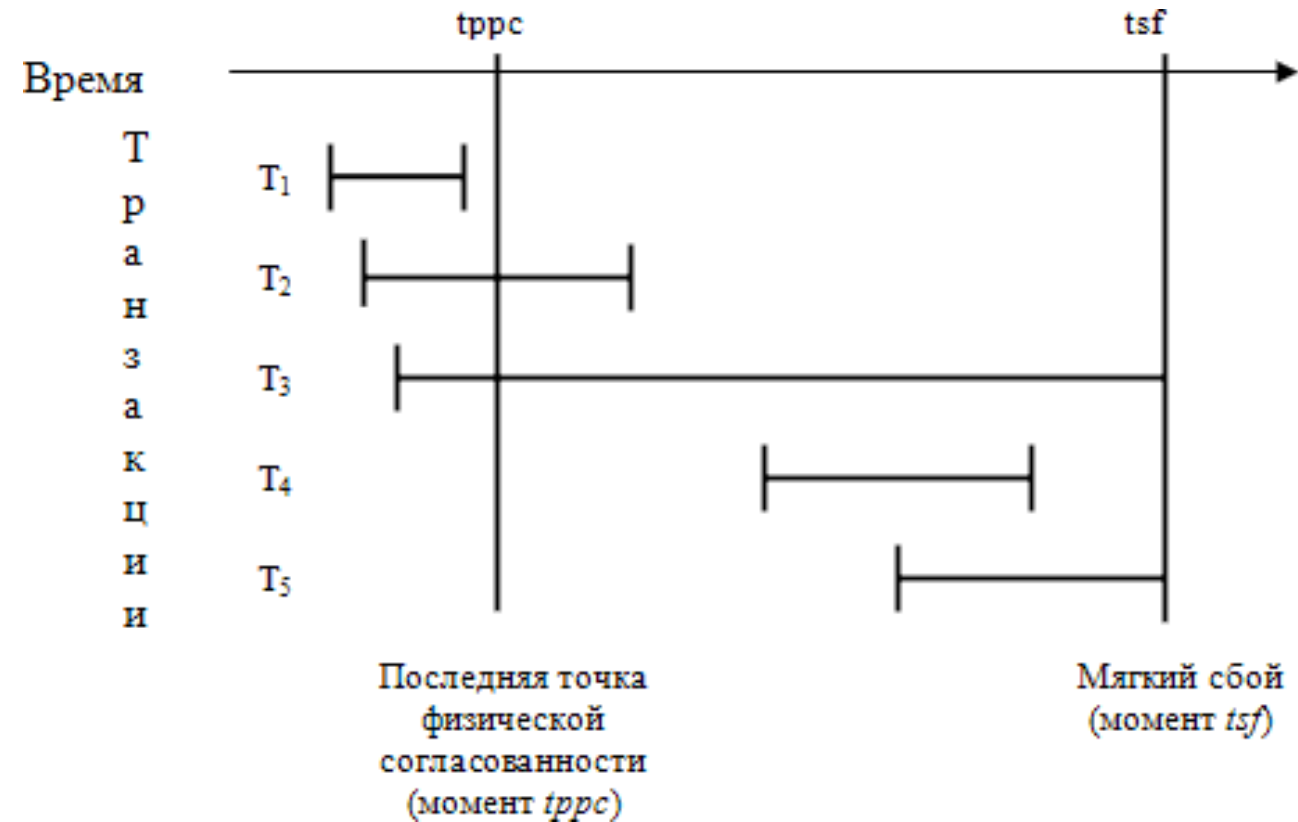
Будем считать, что в журнале отмечаются точки физической согласованности базы данных – моменты времени, в которые во внешней памяти содержатся согласованные результаты операций, завершившихся до соответствующего момента времени, и отсутствуют результаты операций, которые не завершились, а буфер журнала вытолкнут во внешнюю память. Назовем такие точки ppc (point of physical consistency).

ОБРАБОТКА ТРАНЗАКЦИЙ ПРИ МЯГКОМ СБОЕ (1/2)



➔ Для транзакции T_1 никаких действий производить не требуется. Она закончилась до момента $tprc$, и все ее результаты гарантированно отражены во внешней памяти базы данных.

➔ Для транзакции T_2 нужно повторно выполнить (*redo*) последовательность операций, которые выполнялись после установки точки физически согласованного состояния в момент $tprc$. Действительно, во внешней памяти полностью отсутствуют следы операций, которые выполнялись в транзакции T_2 после момента $tprc$. Следовательно, повторное прямое (по смыслу и хронологии) выполнение операций транзакции T_2 корректно и приведет к логически согласованному состоянию базы данных. (Поскольку транзакция T_2 успешно завершилась до момента мягкого сбоя tfs , в журнале содержатся записи обо всех изменениях базы данных, произведенных этой транзакцией.)



ОБРАБОТКА ТРАНЗАКЦИЙ ПРИ МЯГКОМ СБОЕ (2/2)



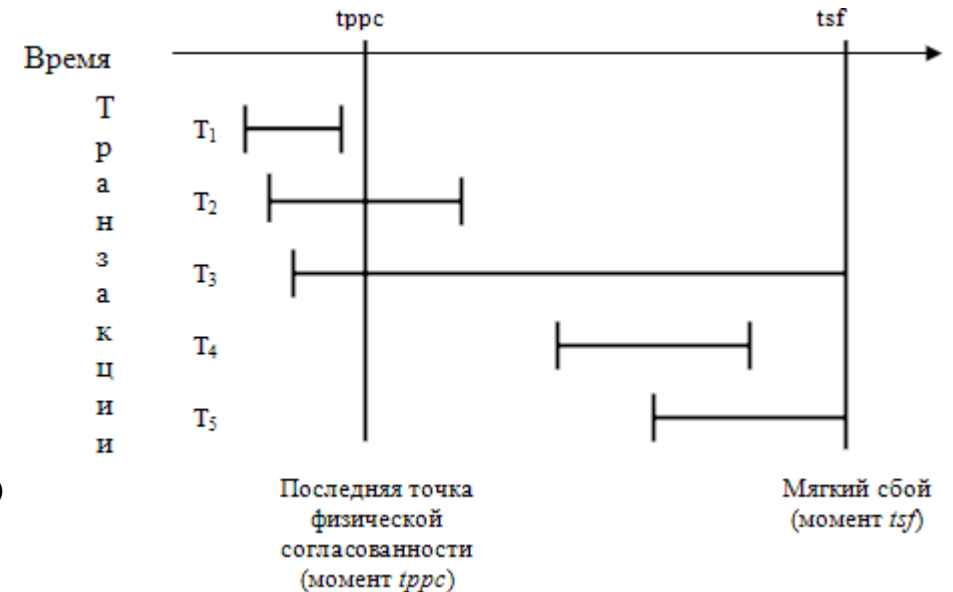
Для транзакции T_3 нужно выполнить в обратном направлении (undo) ту часть операций, которую она успела выполнить до момента $tprc$. Действительно, во внешней памяти базы данных полностью отсутствуют результаты операций T_3 , которые были выполнены после момента $tprc$. С другой стороны, во внешней памяти гарантированно присутствуют результаты операций T_3 , которые были выполнены до момента $tprc$. Следовательно, обратное выполнение (по смыслу и хронологии) операций T_3 корректно и приведет к согласованному состоянию базы данных. (Поскольку транзакция T_3 не завершилась к моменту мягкого сбоя tfs , при восстановлении необходимо устранить все последствия ее выполнения.)



Для транзакции T_4 , которая успела начаться после момента $tprc$ и закончиться до момента мягкого сбоя tfs , нужно произвести полное повторное выполнение операций в прямом направлении. (Поскольку транзакция T_4 успешно завершилась до момента мягкого сбоя tfs , в журнале содержатся записи обо всех изменениях базы данных, произведенных этой транзакцией).



Наконец, для транзакции T_5 , начавшейся после момента $tprc$ и не успевшей завершиться к моменту мягкого сбоя tfs , никаких действий предпринимать не требуется. Результаты операций этой транзакции полностью отсутствуют во внешней памяти базы данных.

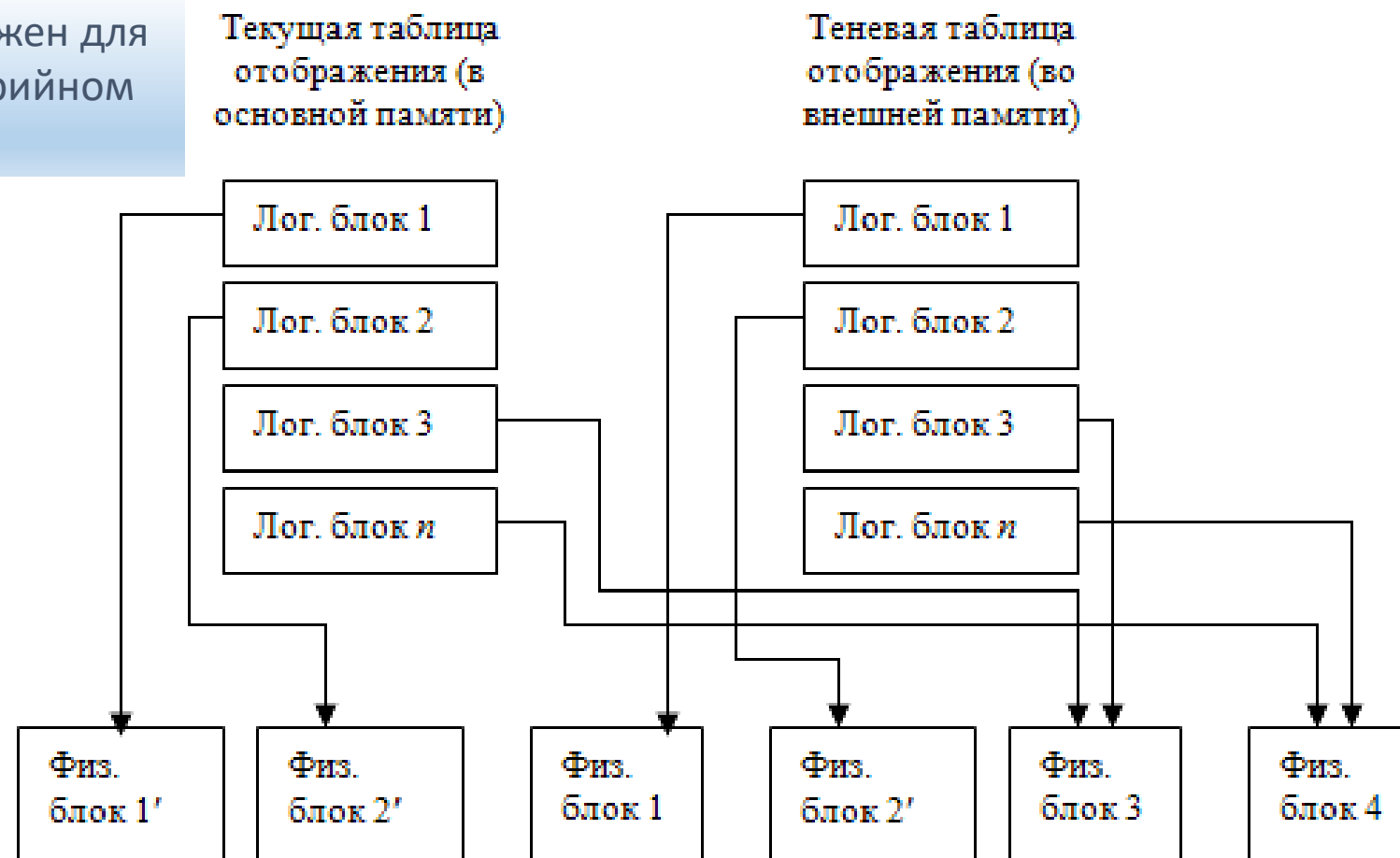


ТЕНЕВОЙ МЕХАНИЗМ ВОССТАНОВЛЕНИЯ ФАЙЛОВ



Теневой механизм был изначально предложен для поддержания целостности файлов при аварийном отключении питания компьютера.

При открытии файла таблица отображения номеров его логических блоков в адреса физических блоков внешней памяти считывается в оперативную память. При модификации любого блока файла во внешней памяти выделяется новый блок. При этом текущая таблица отображения (в основной памяти) изменяется, а теньевая остается неизменной. Если во время работы с открытым файлом происходит сбой, во внешней памяти автоматически сохраняется состояние файла до его открытия.



Для явного восстановления файла достаточно повторно считать в основную память теньевую таблицу отображения.

В контексте базы данных теневой механизм используется следующим образом. Периодически выполняются операции установки точки физической согласованности базы данных. При выполнении этой операции все логические операции завершаются, все страницы буферного пула базы данных, содержимое которых отличается от содержимого соответствующих блоков внешней памяти, выталкиваются. Теневая таблица отображения файлов (сегментов) базы данных заменяется текущей таблицей отображения (правильнее сказать, текущая таблица отображения записывается на место теневой).

ПРОБЛЕМА С ТЕНЕВОЙ ТАБЛИЦЕЙ ОТОБРАЖЕНИЯ

Проблема состоит в том, что в любой момент времени теневая таблица отображения должна быть корректной, т.е. соответствовать некоторому ранее зафиксированному физически целостному состоянию базы данных.

Для этого **необходимо обеспечить атомарность операции замены теневой таблицы отображения**. В общем случае таблица отображения может занимать несколько блоков внешней памяти, и для записи текущей таблицы отображения на место теневой таблицы в этом случае потребуется несколько обменов с дисками. Если в промежутке между этими обменами возникнет мягкий сбой, то будет благополучно утрачена текущая таблица отображения и безнадежно испорчена теневая таблица, т.е. мы просто лишимся возможности восстанавливаться за счет использования последнего физически согласованного состояния базы данных.

- ❑ Во внешней памяти поддерживаются две области хранения таблицы отображения файлов (будем называть их областями А и В).
- ❑ Кроме того, в отдельном блоке внешней памяти хранится флаг F , показывающий, какая из этих областей в данный момент содержит действующую теневую таблицу отображения (назовем соответствующие значения флага F_A и F_B).
- ❑ Тогда, если сохраненным во внешней памяти значением флага является F_A , то текущая таблица отображения записывается в область В. Если эта операция выполняется успешно, то в блок флага записывается значение F_B . Считается, что операция записи одного блока на диск является атомарной. Если эта операция заканчивается успешно, это означает, что новая теневая таблица отображения хранится в области В. Если же запись текущей таблицы отображения в область В не удалась, или если не выполнялась операция записи блока с флагом F , то продолжает действовать старая теневая таблица отображения.
- ❑ Восстановление хронологически последнего сохраненного физически согласованного состояния базы данных происходит мгновенно: текущая таблица отображения заменяет теневой таблицей (при восстановлении просто считывается действующая теневая таблица отображения). Все проблемы восстановления решаются, но за счет слишком большого перерасхода внешней памяти. В пределе может потребоваться вдвое больше внешней памяти, чем реально нужно для хранения базы данных.

ЖУРНАЛИЗАЦИЯ ПОСТРАНИЧНЫХ ИЗМЕНЕНИЙ



Возможен другой подход, при использовании которого наряду с логической журнализацией операций изменения базы данных производится журнализация постраничных изменений.

- ➔ Первый этап восстановления после мягкого сбоя состоит в постраничном откате невыполненных логических операций. Подобно тому, как это делается с логическими записями по отношению к транзакциям, последней записью о постраничных изменениях от одной логической операции является запись о конце операции.
- ➔ При выполнении логической операции обновления базы данных может изменяться несколько блоков базы данных. Для обеспечения возможности отката отдельной операции приходится до конца операции монополюно блокировать все страницы буферного пула базы данных, содержащие копии изменяемых этой операцией блоков базы данных.

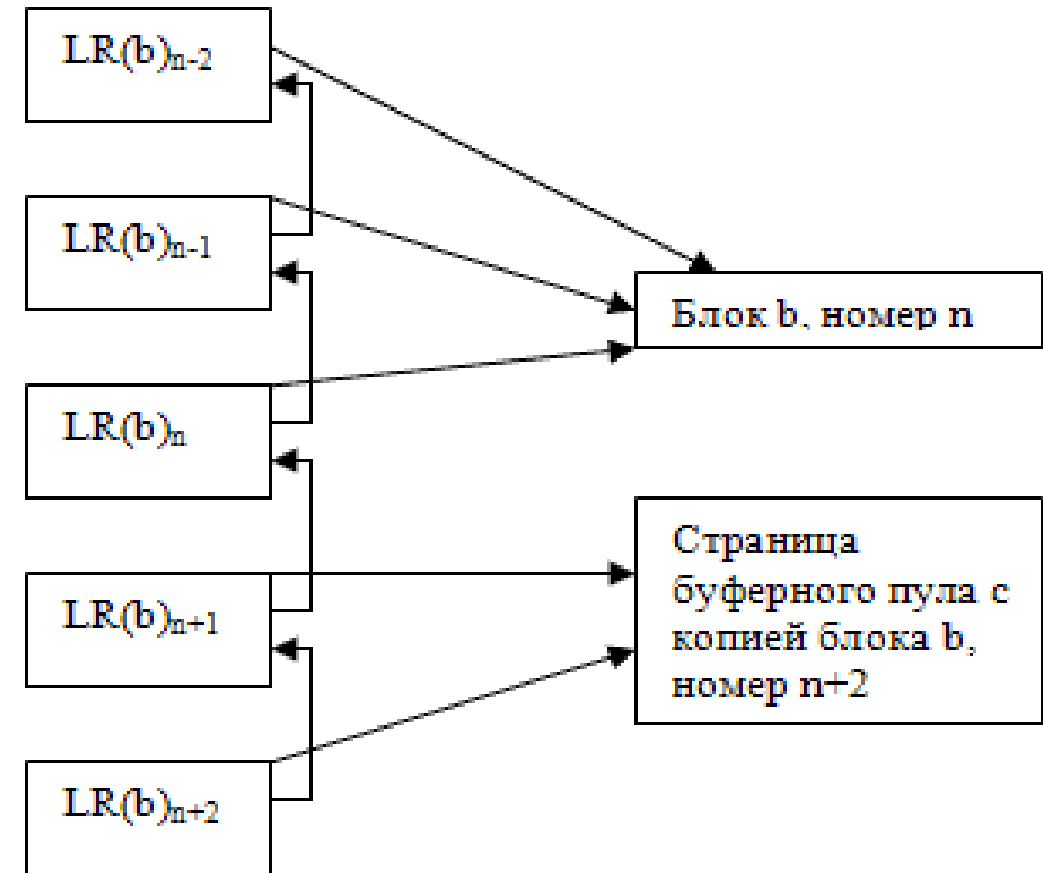
Чтобы распознать, нуждается ли страница внешней памяти базы данных в восстановлении, при выталкивании любой страницы из буферного пула основной памяти в нее помещается номер последней записи о постраничном изменении этой страницы. Этот же номер запоминается в самой записи. Тогда, чтобы понять, нужно ли применить данную запись о постраничном изменении соответствующего блока внешней памяти для восстановления состояния этого блока, требуется всего лишь сравнить номер, содержащийся в этом блоке, с номером, содержащимся в журнальной записи. Если в блоке содержится номер, меньший номера журнальной записи, то это означает, что буферная страница, в которой выполнялось соответствующее изменение, не была к моменту мягкого сбоя вытолкнута во внешнюю память, и применять данную запись для восстановления соответствующего блока внешней памяти не требуется.

ЖУРНАЛИЗАЦИЯ ПОСТРАНИЧНЫХ ИЗМЕНЕНИЙ: ПРИМЕР

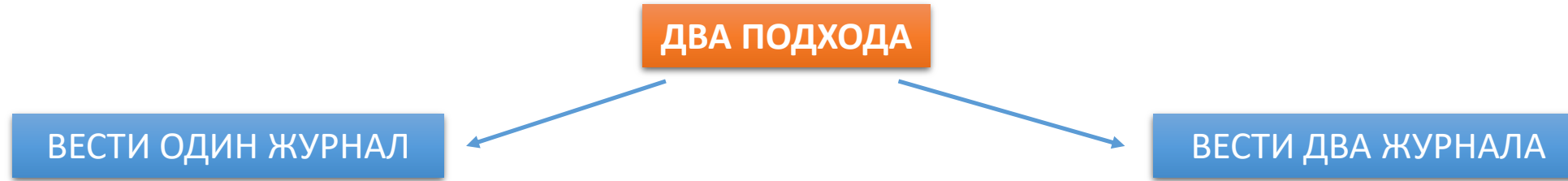


Пусть в рамках какой-то транзакции с блоком b последовательно происходят изменения $LR(b)_{n-2}, \dots, LR(b)_{n+2}$. И тут случился сбой.

В блоке b содержится номер n . Это означает, что в состоянии блока отражены результаты операций изменения блока, соответствующих журнальным записям $LR(b)_n, LR(b)_{n-1}$ и $LR(b)_{n-2}$. Изменения блока, произведенные операциями, которым соответствуют две хронологически последние журнальные записи $LR(b)_{n+1}$ и $LR(b)_{n+2}$, в его состоянии во внешней памяти не отражены, поскольку не было выполнено выталкивание во внешнюю память страницы буферного пула, содержащей копию блока b . Поэтому при восстановлении состояния блока требуется выполнить обратные операции изменения блока b , соответствующие журнальным записям $LR(b)_n, LR(b)_{n-1}$ и $LR(b)_{n-2}$.



ЖУРНАЛИЗАЦИЯ ПОСТРАНИЧНЫХ ИЗМЕНЕНИЙ: ВАРИАЦИИ



Поддерживается общий журнал логических и страничных операций. Естественно, наличие двух видов записей, интерпретируемых абсолютно по-разному, усложняет структуру журнала. Кроме того, записи о страничных изменениях, актуальность которых носит локальный характер, существенно (и не очень осмысленно) увеличивают журнал.

Распространено поддержание отдельного (короткого) журнала страничных изменений. Такой журнал обычно называют **физическим журналом**, поскольку он содержит записи об изменении физических объектов – блоков внешней памяти. В отличие от этого, журнал логических операций принято называть **логическим журналом**, поскольку в нем содержатся записи об операциях над логическими объектами – кортежами.

Обычно ведут два журнала. Далее более подробно именно об этом подходе.

СВОЙСТВА ЛОГИЧЕСКОГО ЖУРНАЛА

Логический журнал должен поддерживать как обратное выполнение журнализованных операций (*undo*), так и их повторное прямое выполнение (*redo*). В отличие от этого, от физического журнала требуется только поддержка обратного выполнения постраничных операций.

Логический журнал обычно начинает заполняться заново только после выполнения операций резервного копирования базы данных или архивирования самого журнала. До этого времени он линейно растет. Понятно, что в любом случае для размещения журнала выделяется внешняя память ограниченного размера. Предельный размер журнала определяется администратором базы данных и должен согласовываться с размером интервала времени, через которое производится резервное копирование базы данных.

ОБРАБОТКА ПЕРЕПОЛНЕНИЯ ЛОГИЧЕСКОГО ЖУРНАЛА

На пути к достижению максимально возможного размера журнала устанавливаются «желтая» и «красная» зоны. Когда записи в журнал достигают «желтой» зоны, выдается предупреждение администратору базы данных и прекращается образование новых транзакций. Если все существующие транзакции завершаются до достижения «красной» зоны, автоматически выполняется архивация базы данных или логического журнала. Если какие-то транзакции не успевают завершиться до достижения «красной» зоны журнала, выполняется их аварийный откат, после чего производится архивация базы данных или журнала. Естественно, размер «желтой» и «красной» зон логического журнала должен устанавливаться администратором базы данных с учетом максимально допустимого числа одновременно существующих транзакций и их возможной протяженности.

Физический журнал существует сравнительно недолгое время (интервал времени между соседними операциями установки точки физической согласованности базы данных) и, как правило, занимает существенно меньшее дисковое пространство, чем логический журнал.

ВЫПОЛНЕНИЕ ОПЕРАЦИИ УСТАНОВКИ ТОЧКИ ФИЗИЧЕСКОЙ СОГЛАСОВАННОСТИ

1. Прекращают инициироваться новые логические операции;
2. после завершения всех выполняемых логических операций происходит выталкивание во внешнюю память всех модифицированных страниц буферного пула;
3. формируется и выталкивается во внешнюю память логического журнала специальная запись о точке физически согласованного состояния;
4. в случае успешного предыдущего действия разрешается инициация новых логических операций, и физический журнал пишется заново.

Предпоследняя операция является атомарной (это опять же запись одного блока на диск): если она успешно выполняется, то при следующем восстановлении после мягкого сбоя будет использоваться новая точка физически согласованного состояния, иначе ситуация воспринимается как мягкий сбой с восстановлением логически согласованного состояния базы данных от предыдущей точки физически согласованного состояния (с оповещением об этом администратора базы данных).

Жесткий сбой – утрата внешней памяти, при этом буфер оперативной памяти может не пострадать.

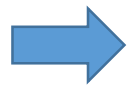
Для **восстановления последнего согласованного состояния базы данных** после жесткого сбоя журнала изменений базы данных явно недостаточно.

Самый простой способ восстановления основывается на использовании логического журнала и архивной копии базы данных.

Восстановление начинается с обратного копирования (на исправный носитель) базы данных из архивной копии. Затем для всех закончившихся транзакций выполняется *redo*, т.е. операции повторно выполняются в прямом смысле.

Более точно, происходит следующее:

- по журналу в прямом направлении выполняются все операции;
- для транзакций, которые не закончились к моменту сбоя, выполняется откат.



Поскольку жесткий сбой не сопровождается утратой буферов оперативной памяти, можно восстановить базу данных до такого уровня, чтобы можно было продолжить даже выполнение незавершенных транзакций. Но обычно это не делается, потому что восстановление после жесткого сбоя – это достаточно длительный процесс.

Хотя к ведению журнала предъявляются особые требования по части надежности, в принципе возможна и его утрата. Тогда единственным способом восстановления базы данных является возврат к архивной копии. Конечно, в этом случае не удастся получить последнее согласованное состояние базы данных, но это лучше, чем ничего.

Вместо базы данных можно архивировать сам журнал.

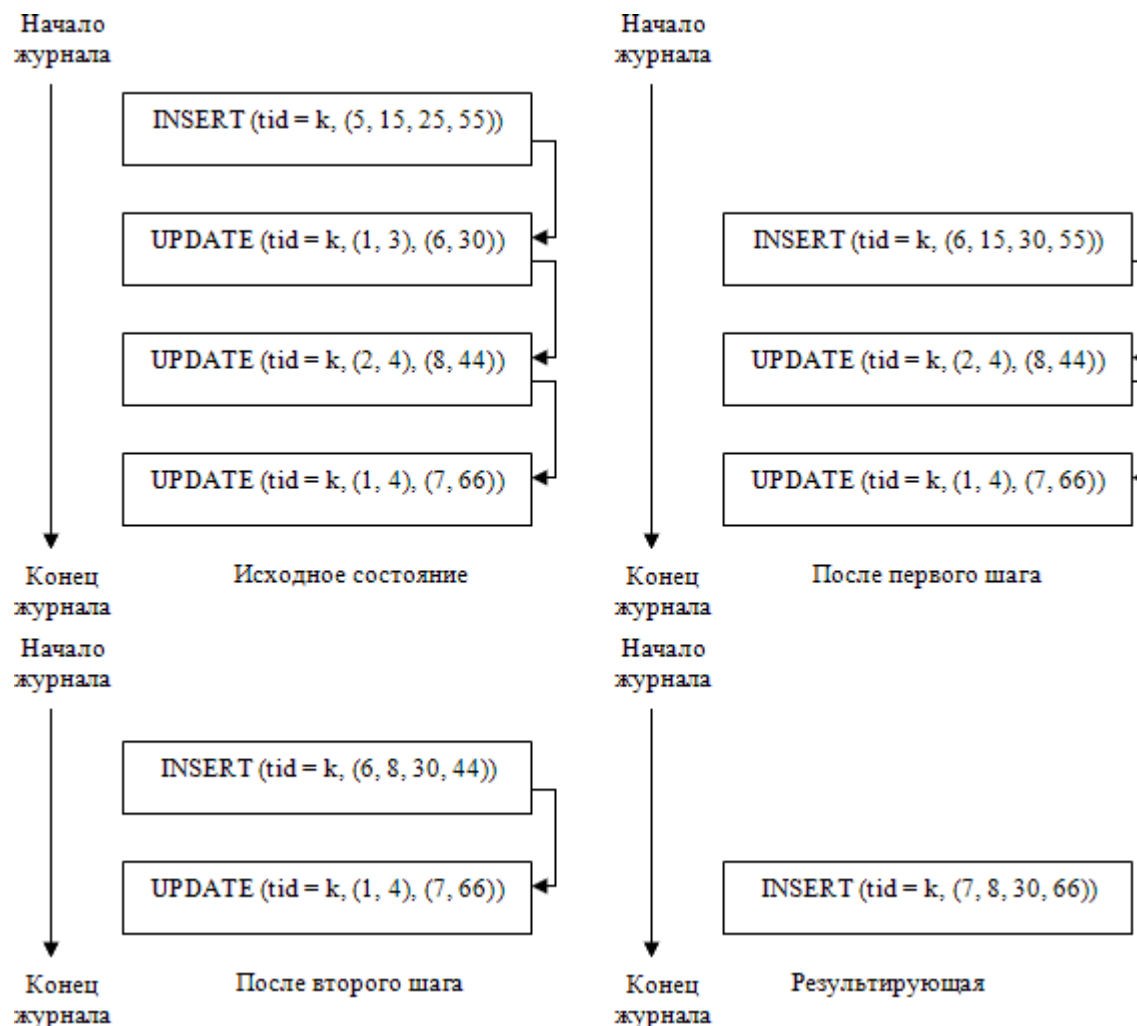
В пределе для полного восстановления базы данных после жесткого сбоя достаточно иметь исходную архивную копию базы данных, последовательность архивных копий журналов и последний логический журнал.

Архивированный логический журнал можно сжимать.

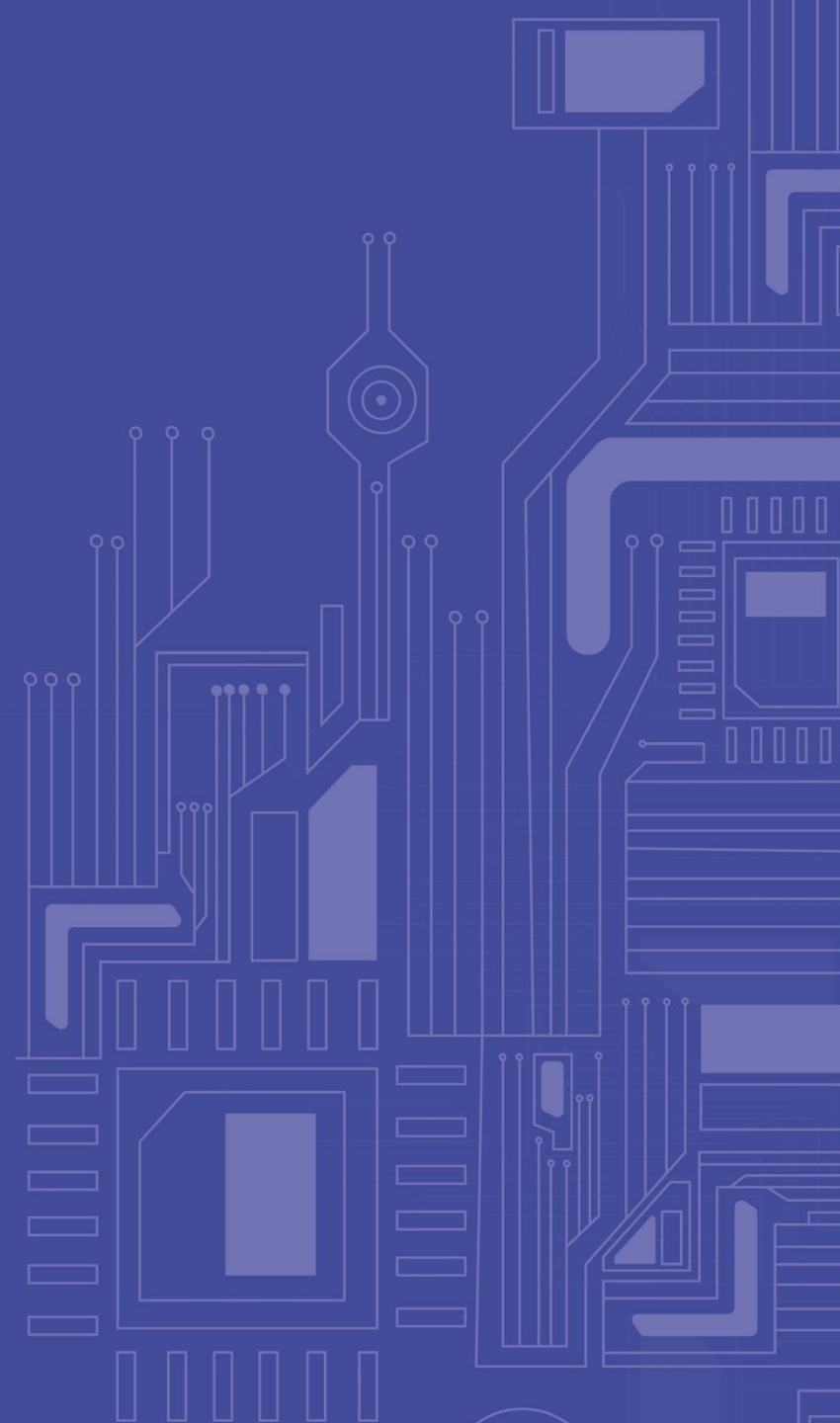
Для этого для каждого объекта базы данных нужно найти последовательность журнальных записей, относящихся к этому объекту, в хронологическом порядке и заменить их одной записью, соответствующей операции над объектом, результат которой эквивалентен результату последовательного выполнения журнализованных операций из построенной последовательности.



Таким образом, для восстановления после жесткого сбоя можно воспользоваться исходной архивной копией, одним сжатым архивным журналом и последним логическим журналом.



ВВЕДЕНИЕ В SQL



Язык SQL, предназначенный для взаимодействия с базами данных, появился в середине 70-х гг. (первые публикации датируются 1974 г.) и был разработан в компании IBM в рамках проекта экспериментальной реляционной СУБД System R. Исходное название языка SEQUEL (Structured English Query Language) только частично отражало суть этого языка. Конечно, язык был ориентирован главным образом на удобную и понятную пользователям формулировку запросов к реляционным БД.

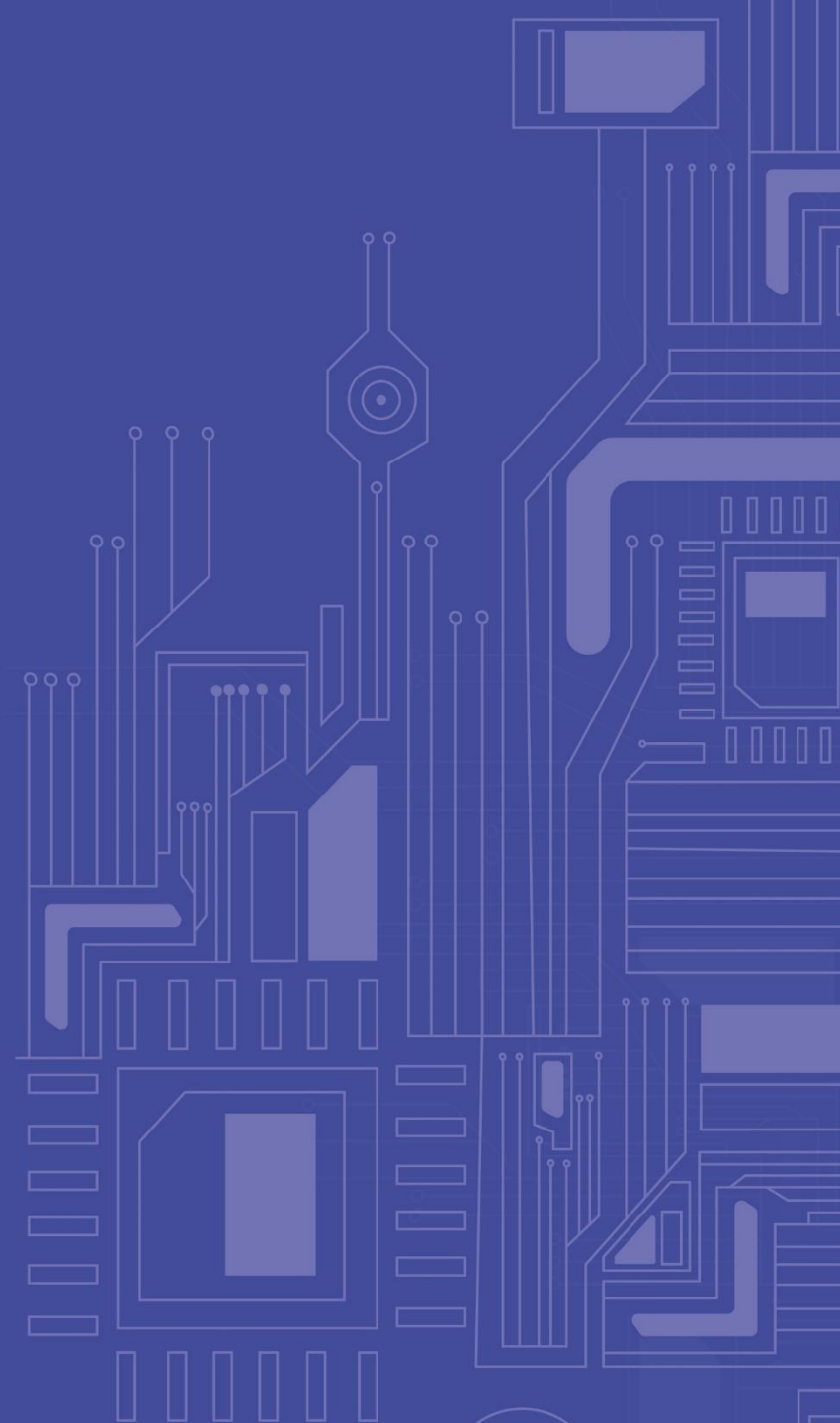
Возможности первого SEQUEL

- средства определения и манипулирования схемой БД;
- средства определения ограничений целостности и триггеров;
- средства определения представлений БД;
- средства определения структур физического уровня, поддерживающих эффективное выполнение запросов;
- средства авторизации доступа к отношениям и их полям;
- средства определения точек сохранения транзакции и выполнения фиксации и откатов транзакций.

В языке отсутствовали средства явной синхронизации доступа к объектам БД со стороны параллельно выполняемых транзакций: с самого начала предполагалось, что необходимую синхронизацию неявно выполняет СУБД.

Год	Название	Другое название	Изменения
1999	SQL:1999	SQL3	Добавлена поддержка регулярных выражений, рекурсивных запросов, поддержка триггеров, базовые процедурные расширения, нескаларные типы данных и некоторые объектно-ориентированные возможности.
2003	SQL:2003		Введены расширения для работы с XML-данными, оконные функции (применяемые для работы с OLAP-базами данных), генераторы последовательностей и основанные на них типы данных.
2006	SQL:2006		Функциональность работы с XML-данными значительно расширена. Появилась возможность совместно использовать в запросах SQL и XQuery.
2008	SQL:2008		Улучшены возможности оконных функций, устранены некоторые неоднозначности стандарта SQL:2003

ТИПЫ ДАННЫХ SQL



Данные, хранящиеся в столбцах таблиц SQL-ориентированной базы данных, являются типизированными, т. е. представляют собой значения одного из типов данных, predetermined в языке SQL или определяемых пользователями путем применения соответствующих средств языка. Для этого при определении таблицы каждому ее столбцу назначается некоторый тип данных (или домен), и в дальнейшем СУБД должна следить, чтобы в каждом столбце каждой строки каждой таблицы присутствовали только допустимые значения.

КАТЕГОРИИ ТИПОВ ДАННЫХ SQL

- точные числовые типы (exact numerics);
- приближенные числовые типы (approximate numerics);
- типы символьных строк (character strings);
- типы битовых строк (bit strings);
- типы даты и времени (datetimes);
- типы временных интервалов (intervals);
- булевский тип (Booleans);
- типы коллекций (collection types);
- анонимные строчные типы (anonymous row types);
- типы, определяемые пользователем (user-defined types);
- ссылочные типы (reference types)

В столбцах таблиц, определенных на любых типах данных, наряду со значениями этих типов, допускается сохранение неопределенного значения, которое обозначается ключевым словом **NULL**.

- Результатом выражений вида $x \text{ op } \text{NULL}$, $\text{NULL op } x$, NULL op NULL является **NULL** для всех арифметических операций.
- Значением выражений $x \text{ comp_op } \text{NULL}$, $\text{NULL comp_op } x$, NULL comp_op NULL для всех операций сравнения является третье логическое значение **unknown**.

К категории *точных числовых типов* в SQL относятся те типы, значения которых точно представляют числа. Типы данных этой категории распадаются на две части: *истинно целые типы* (INTEGER и SMALLINT) и *типы, допускающие наличие дробной части* (NUMERIC и DECIMAL).

Истинно целые типы

- Тип **INTEGER** служит для представления целых чисел. Точность чисел (число сохраняемых бит) определяется в реализации. При определении столбца данного типа достаточно указать просто INTEGER.
- Тип **SMALLINT** также служит для представления целых чисел. Точность определяется в реализации, но она не должна быть больше точности типа INTEGER. При определении столбца указывается просто SMALLINT.
- Литералы типов целых чисел представляются в виде строк символов, изображающих десятичные числа; в начале строки могут присутствовать символы «+» или «-» (если символ знака отсутствует, подразумевается «+»). Примеры литералов типов INTEGER и SMALLINT: 1826545, 876.

Точные типы, допускающие наличие дробной части

- Тип **NUMERIC**. На самом деле, это не просто тип данных, а параметризуемый тип. При определении столбца можно указать спецификацию `NUMERIC (p, s)`, где `p` и `s` – литералы истинно целого типа, и `p` задает *точность значений* (число сохраняемых бит), а `s` – шкалу (число десятичных цифр в дробной части). Задаваемая шкала не должна быть отрицательной и не должна превышать значение точности. При определении столбца можно использовать сокращенные формы спецификации типа – `NUMERIC` и `NUMERIC (p)`. Первая форма предполагает использование точности, определяемое по умолчанию в реализации, и шкалы, равной нулю, а вторая – использование заданной точности и шкалы, равной нулю. Допустимые диапазоны значений `p` и `s` определяются в реализации.
- Тип **DECIMAL**. Этот тип аналогичен типу `NUMERIC`. Отличие состоит в том, что если при определении столбца типа `DECIMAL` задается точность `p`, то на самом деле используется точность `m`, определяемая в реализации, такая, что $m > p$. Шкала всегда устанавливается такой, как явно или неявно (по умолчанию) задается. При указании типа столбца можно использовать спецификации `DECIMAL`, `DECIMAL (p)` и `DECIMAL (p, s)`.
- Литералы типов точных чисел, допускающих наличие дробной части, представляются в виде строк символов, изображающих десятичные числа, в начале которых могут присутствовать символы «+» или «-» (если символ знака отсутствует, подразумевается «+»), а внутри последовательности цифр может присутствовать символ «.». Примеры литералов типов `NUMERIC` и `DECIMAL`: 125, 26.36.

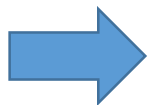
- Тип **REAL**. Значения типа соответствуют числам с плавающей точкой одинарной точности. Точность определяется в реализации, но обычно совпадает с точностью одинарной плавающей арифметики, поддерживаемой на аппаратной платформе, которая используется реализацией. При определении столбца указывается просто REAL.
- Тип **DOUBLE PRECISION**. Точность значений этого типа определяется в реализации, но она должна быть больше точности типа REAL. Обычно приближенным числам SQL с двойной точностью соответствуют поддерживаемые аппаратурой числа с плавающей точкой двойной точности. При определении столбца указывается просто DOUBLE PRECISION.
- Тип **FLOAT**. Это параметризуемый тип, значение параметра p которого задает необходимую точность значений. Требуется, чтобы реально обеспечиваемая реализацией точность значений была не меньше p . Допустимый диапазон значений параметра p определяется в реализации. При определении столбца можно указать либо **FLOAT (p)**, либо просто **FLOAT**. В последнем случае подразумевается точность, определяемая реализацией по умолчанию.
- Литералы приближенных числовых типов представляются в виде литерала точного числового типа, за которым могут следовать символ «E» и литерал целого числового типа. Примеры литералов приближенных числовых типов: 123, 123.12, 123E12, 123.12E12. Литеральное выражение xEy представляет значение $x \cdot (10^y)$.

В SQL определены три параметризуемых *типа символьных строк*: CHARACTER (или CHAR), CHARACTER VARYING (или CHAR VARYING, или VARCHAR) и CHARACTER LARGE OBJECT (или CLOB).

- Тип **CHARACTER**. Значениями типа являются символьные строки. Конкретный набор допустимых символов определяется в реализации, но, как правило, включает набор символов ASCII. При определении столбца допускается использование спецификаций CHARACTER (x) и просто CHARACTER. Последний вариант эквивалентен заданию CHARACTER (1). После определения столбца типа CHARACTER (x) СУБД будет резервировать место для хранения x символов этого столбца во всех строках соответствующей таблицы. Если, например, определен столбец типа CHARACTER (8), и в некоторой строке таблицы в него заносится символьная строка длиной пять символов, то реально будут храниться восемь символов, последние три из которых будут пробелами.
- Тип **CHARACTER VARYING**. При определении столбца допускается использование спецификаций CHARACTER VARYING (x) и просто CHARACTER VARYING. Последний вариант эквивалентен заданию CHARACTER VARYING (1). Если в некоторой таблице определяется столбец типа CHARACTER VARYING (x), то в каждой строке этой таблицы значения данного столбца будут занимать ровно столько места, сколько требуется для сохранения соответствующей символьной строки (но ни одна такая строка не может состоять более чем из x символов).
- Тип **CHARACTER LARGE OBJECT**. Этот тип данных предназначен для определения столбцов, хранящих большие и разные по размеру группы символов. При определении столбца задается спецификация CLOB (z), где z задает максимальный размер соответствующей группы символов. Максимально возможное значение параметра z определяется в реализации, но, очевидно, что оно должно быть существенно больше максимально возможного значения параметра x, присутствующего в типах CHAR и CHAR VARYING.

Определен ряд операций, которые можно выполнять над символьными строками. Перечислим некоторые из них.

- Операция конкатенации (обозначается в виде «`||`») возвращает символьную строку, произведенную путем соединения строк-операндов в том порядке, в каком они заданы.
- Функция выделения подстроки (**SUBSTRING**) принимает три аргумента – строку, номер начальной позиции и длину – и возвращает строку, выделенную из строки-аргумента в соответствии со значениями двух последних параметров.
- Функция **UPPER** возвращает строку, в которой все строчные буквы строки-аргумента заменяются прописными. Функция **LOWER**, наоборот, заменяет в заданной строке все прописные буквы строчными.
- Функция определения длины (**CHARACTER_LENGTH**, **OCTET_LENGTH**, **BIT_LENGTH**) возвращает длину заданной символьной строки в символах, октетах или битах (в зависимости от вида вычисляющей функции) в виде целого числа.
- Функция определения позиции (**POSITION**) определяет первую позицию в строке *S*, с которой в нее входит заданная строка *S1* (если не входит, то возвращается значение нуль).



Литералы типов символьных строк представляются в виде последовательностей символов, заключенных в одинарные или двойные кавычки. В первом случае среди набора символов литерала допускается наличие символов двойной кавычки, а во втором – символов одинарной кавычки. Примеры литералов символьных строк: `'ABCDEF'`, `'Ab"Ctd'`, `"Fbcdef"`, `"ab'cdtF"`.

В SQL определены три параметризуемых *типа битовых строк*: BIT, BIT VARYING и BINARY LARGE OBJECT (или BLOB).

- Тип **BIT**. Значениями типа являются битовые строки. При определении столбца допускается использование спецификаций BIT (x) и просто BIT. Последний вариант эквивалентен заданию BIT (1). После определения столбца типа BIT (x) СУБД будет резервировать место для хранения x бит этого столбца во всех строках соответствующей таблицы.
- Тип **BIT VARYING**. При определении столбца допускается использование только спецификации без умолчания вида BIT VARYING (x), где значение x определяет максимальную длину битовой строки, которую можно хранить в данном столбце.
- Тип **BINARY LARGE OBJECT**. Этот тип данных предназначен для определения столбцов, хранящих большие и разные по размеру группы байтов. При определении столбца задается спецификация **BLOB** (z), где z задает максимальный размер соответствующей группы байтов. С технической точки зрения типы CLOB и BLOB очень похожи. Их разделение требуется для того, чтобы подчеркнуть, что значения типа CLOB состоят из символов (в частности, в них может осмысленно производиться текстовый поиск), а значения типа BLOB состоят из произвольных байтов, не обязательно кодирующих символы.

Над битовыми строками определен ряд операций. Некоторые из них мы рассмотрим.

- Битовая конкатенация (обозначается в виде `||`), которая возвращает результирующую битовую строку, полученную путем конкатенации строк-аргументов в том порядке, в котором они заданы.
- Функция извлечения подстроки из битовой строки. Синтаксис и семантика этой функции идентичны синтаксису и семантике функции **SUBSTRING** для символьных строк, за исключением того, что первый аргумент и возвращаемое значение являются битовыми строками.
- Функция определения длины (**OCTET_LENGTH**, **BIT_LENGTH**) возвращает длину заданной битовой строки в октетах или битах в зависимости от выбранной функции.
- Функция определения позиции (**POSITION**) определяет первую позицию в битовой строке *S*, с которой в нее входит строка *S1*. Если строка *S1* не входит в строку *S*, возвращается значение нуль.

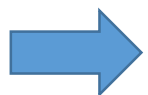


Литералы типов битовых строк представляются как заключенные в одинарные кавычки последовательности символов «0» и «1», предваряемые символом «B»; или предваряемые символом «X» последовательности символов, которые изображают шестнадцатеричные цифры (за цифрой «9» следуют «A», «B», «C», «D», «E» и «F»).
Примеры литералов типов битовых строк: `B'011100111100011111111'`, `X'78FBCD0012FFFA'`.

Возможность сохранения в базе данных информации о дате и времени очень важна с практической точки зрения. Достаточно вспомнить взбудоражившую весь мир «проблему 2000 года», одним из основных источников которой было некорректное хранение дат в базах данных. В стандарте SQL поддержке средств работы с датой и временем уделяется большое внимание. В частности, поддерживаются специальные «темпоральные» типы данных DATE, TIME, TIMESTAMP, TIME WITH TIME ZONE и TIMESTAMP WITH TIME ZONE.

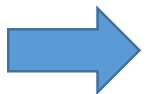
Тип даты

- Тип DATE. Значения этого типа состоят из компонентов-значений года, месяца и дня некоторой даты. Значение года состоит из четырех десятичных цифр и соответствует летоисчислению от Рождества Христова до 9999 г. Значение месяца состоит из двух десятичных цифр и варьируется от 01 до 12. Значение номера дня месяца состоит из двух десятичных цифр и варьируется от 01 до 31, хотя значение месяца даты может накладывать ограничения на возможность использования значений дня месяца 29, 30 и 31. В стандарте SQL не накладываются какие-либо ограничения на внутренний способ представления дат, используемый в реализации. При определении столбца типа DATE указывается просто DATE.



Литералы типа DATE представляются в виде строки «'yyyy-mm-dd'», где символы y, m и d должны изображать десятичные числа. Например, литерал DATE '1949-04-08' представляет дату 8 апреля 1949 г.

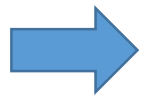
- Тип **TIME**. Значения этого параметризованного типа состоят из компонентов-значений часа, минуты и секунды некоторого времени суток. Значение часа состоит ровно из двух десятичных цифр и варьируется от 00 до 23. Значение минуты состоит из двух десятичных цифр и варьируется от 00 до 59. Основное значение секунды также состоит из двух цифр, но может включать дополнительные цифры, представляющие доли секунды. Так что в целом значение секунды варьируется от 00 до 61.999... В значении времени присутствуют две лишние секунды, поскольку Всемирная служба времени иногда добавляет две секунды к последней минуте года для синхронизации мирового времени с реальным. Решение о поддержке этих «високосных» секунд принимается на уровне реализации. Число цифр в доле секунды также определяется в реализации. В стандарте требуется только то, чтобы это число было не меньше шести. При определении столбца типа TIME может указываться TIME (p) (значение p задает точность долей секунды) или просто TIME (в этом случае доли секунды не учитываются).



Литералы типа TIME представляются в виде строки TIME 'hh:mm:ss:f...f', где символы h, m, s и f должны изображать десятичные числа. Например, литерал TIME '16:33-20:333' представляет время суток 16 часов 33 минуты 20 и 333 тысячных секунды.

Типы временной метки

- Тип **TIMESTAMP**. Значения этого параметризованного типа состоят из компонентов — значений года, месяца и дня некоторой даты, а также компонентов — значений часа, минуты и секунды некоторого времени суток. Число десятичных цифр в значениях-компонентах и ограничения этих значений такие же, как у значений типов DATE и TIME. При определении столбца типа TIMESTAMP может указываться TIMESTAMP (p) (значение p задает точность долей секунды) или просто TIMESTAMP (в этом случае, в отличие от типа данных TIME, по умолчанию принимается, что в доли секунды используются шесть десятичных цифр).



Литералы типа TIMESTAMP представляются в виде строки `TIMESTAMP 'yyyy-mm-dd hh:mm:ss:f...f'`, где символы y, m, d, h, m, s и f должны изображать десятичные числа.

Типы времени и временной метки с временной зоной

- Тип **TIME WITH TIME ZONE**. Этот тип данных похож на тип TIME с тем лишь отличием, что значения типа TIME WITH TIME ZONE включают дополнительный компонент — значение, характеризующее смещение соответствующего времени относительно гринвичского времени (теперь его называют UTC – universal time coordinated). Деталей представления этого дополнительного компонента мы касаться не будем.
- Тип **TIMESTAMP WITH TIME ZONE**. Этот тип данных отличается от типа TIMESTAMP тем, что значения типа TIMESTAMP WITH TIME ZONE включают дополнительный компонент-значение, характеризующее смещение соответствующего времени относительно гринвичского.

ТИПЫ ВРЕМЕННЫХ ИНТЕРВАЛОВ (1/2)



Вообще говоря, **временным интервалом** называется разность между двумя значениями даты или времени. В SQL определены две категории *типов временных интервалов*: «год-месяц» и «день-время суток». Временные интервалы языка SQL не привязываются к начальному и/или конечному значению даты/времени, а описывают только протяженность во времени. В общем случае при определении столбца типа временного интервала указывается INTERVAL start (p) [TO end (q)], где в качестве «start» и «end» могут задаваться YEAR, MONTH, DAY, HOUR, MINUTE и SECOND.

Параметр p задает требуемую точность лидирующего поля интервала (число десятичных цифр). Параметр q может задаваться только в том случае, когда в качестве end используется SECOND, и указывает точность долей секунды. Если говорить более точно, возможны следующие вариации типов временных интервалов.

- **Типы категории «день-время суток».** При определении столбца можно использовать комбинации, представленные справа. Если значение параметра p не указывается явно, по умолчанию принимается его значение «2». Значением параметра q по умолчанию является «6».

```
INTERVAL DAY (p),  
INTERVAL DAY,  
INTERVAL DAY (p) TO HOUR,  
INTERVAL DAY TO HOUR,  
INTERVAL DAY (p) TO MINUTE,  
INTERVAL DAY TO MINUTE,  
INTERVAL DAY (p) TO SECOND (q),  
INTERVAL DAY TO SECOND (q),  
INTERVAL DAY (p) TO SECOND,  
INTERVAL DAY TO SECOND,  
INTERVAL HOUR (p),  
INTERVAL HOUR, INTERVAL HOUR (p) TO MINUTE,  
INTERVAL HOUR TO MINUTE,  
INTERVAL HOUR (p) TO SECOND (q),  
INTERVAL HOUR TO SECOND (q),  
INTERVAL HOUR TO SECOND,  
INTERVAL MINUTE (p),  
INTERVAL MINUTE,  
INTERVAL MINUTE (p) TO SECOND (q),  
INTERVAL MINUTE TO SECOND (q),  
INTERVAL MINUTE (p) TO SECOND,  
INTERVAL MINUTE TO SECOND,  
INTERVAL SECOND (p, q),  
INTERVAL SECOND (p),  
INTERVAL SECOND.
```

ТИПЫ ВРЕМЕННЫХ ИНТЕРВАЛОВ (2/2)

- Типы категории «год-месяц». Можно определить столбцы следующих типов: INTERVAL YEAR, INTERVAL YEAR (p) (значения этих типов – временные интервалы в годах), INTERVAL MONTH, INTERVAL MONTH (p) (значения этих типов – временные интервалы в месяцах), INTERVAL YEAR TO MONTH, INTERVAL YEAR (p) TO MONTH (значения этих типов – временные интервалы в годах и месяцах). Если значение параметра p не указывается явно, по умолчанию принимается его значение «2».
- ➔ Пример литерала одной из разновидностей типа INTERVAL: INTERVAL '10:20' MINUTE TO SECOND – временной интервал в 10 минут и 20 секунд.

Над значениями темпоральных типов могут выполняться арифметические операции, смысл которых определяется следующей таблицей:

Тип первого операнда	Операция	Тип второго операнда	Тип результата
Datetime	-	Datetime	Interval
Datetime	+ ИЛИ -	Interval	Datetime
Interval	+	Datetime	Datetime
Interval	+ ИЛИ -	Interval	Interval
Interval	* ИЛИ /	Numeric	Interval
Numeric	*	Interval	Interval



БУЛЕВСКИЙ ТИП ДАННЫХ

При определении столбца булевского типа указывается просто спецификация **BOOLEAN**. Булевский тип состоит из трех значений: true, false и unknown. (соответствующие литералы обозначаются TRUE, FALSE и UNKNOWN). Поддерживается возможность построения булевских выражений, которые вычисляются в трехзначной логике.

Таблицы истинности основных логических операций

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

NOT	
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

Начиная с SQL:1999, в языке поддерживается возможность использования типов данных, значения которых являются коллекциями значений некоторых других типов. Обычно под термином *коллекция* понимается одно из следующих образований: массив, список, множество и мультимножество. В варианте SQL:1999, принятом в 1999 г., были специфицированы только типы массивов. В стандарте SQL:2003 появилась спецификация типа мультимножества.

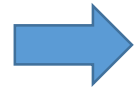
Типы массивов

Любой возможный *тип массива* получается путем применения конструктора типов **ARRAY**. При определении столбца, значения которого должны принадлежать некоторому типу массива, используется конструкция **dt ARRAY [mc]**, где **dt** специфицирует некоторый допустимый в SQL тип данных, а **mc** является литералом некоторого точного числового типа с нулевой длиной шкалы и определяет максимальное число элементов в значении типа массива.

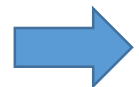
Элементам каждого значения типа массива соответствуют их порядковые номера, называемые индексами. Значение индекса всегда должно принадлежать отрезку [1, **mc**]. Значениями типа массива **dt ARRAY [mc]** являются все массивы, состоящие из элементов типа **dt**, максимальное значение индекса которых **cs** не превосходит значения **mc**. При сохранении в базе данных значения типа массива занимает столько памяти, сколько требуется для сохранения **cs** элементов.

Основными операциями над массивами являются выборка значения элемента массива по его индексу, изменение некоторого элемента массива или массива целиком и конкатенация (сцепление) двух массивов. Кроме того, для любого значения типа массива можно узнать значение его **cs**.

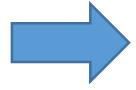
Типы мультимножеств



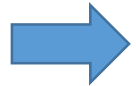
При определении столбца таблицы *типа мультимножеств* используется конструкция **dt MULTISSET**, где **dt** задает тип данных элементов конструируемого типа мультимножеств. Значениями типа мультимножеств являются мультимножества, т. е. неупорядоченные коллекции элементов одного и того же типа, среди которых допускаются дубликаты. Например, значениями типа **INTEGER MULTISSET** являются мультимножества, элементами которых — целые числа. Примером такого значения может быть мультимножество {12, 34, 12, 45, -64}.



В отличие от массива, мультимножество является неограниченной коллекцией; при конструировании типа мультимножеств не указывается предельная кардинальность значений этого типа. Однако это не означает, что возможность вставки элементов в мультимножество действительно не ограничена; стандарт всего лишь не требует явного объявления границы. Ситуация аналогична той, которая возникает при работе с таблицами, для которых в SQL не объявляется максимально допустимое число строк.



Для типов мультимножеств поддерживаются операции преобразования типа значения-мультимножества к типу массивов или другому типу мультимножеств с совместимым типом элементов (операция **CAST**), для удаления дубликатов из мультимножества (функция **SET**), для определения числа элементов в заданном мультимножестве (функция **CARDINALITY**), для выборки элемента мультимножества, содержащего в точности один элемент (функция **ELEMENT**). Кроме того, для мультимножеств обеспечиваются операции объединения (**MULTISET UNION**), пересечения (**MULTISET INTERSECT**) и определения разности (**MULTISET EXCEPT**). Каждая из операций может выполняться в режиме с сохранением дубликатов (режим **ALL**) или с устранением дубликатов (режим **DISTINCT**).



Расширенные в SQL:2003 возможности работы с типами коллекций являются принципиально важными. Даже при наличии определяемых пользователями типов данных и типов массивов SQL:1999 не предоставлял полных возможностей для преодоления исторически присущего реляционной модели данных вообще и SQL в частности ограничения «плоских таблиц». После появления конструктора типов мультимножеств и устранения ограничений на тип данных элементов коллекции это историческое ограничение полностью ликвидировано. Мультимножество, типом элементов которого является анонимный строчный тип (см. ниже), представляет собой полный аналог таблицы. Тем самым, в базе данных допускается произвольная вложенность таблиц. Возможности выбора структуры базы данных безгранично расширяются.

Анонимный строчный тип данных - это конструктор типов ROW, позволяющий производить безымянные типы строк (кортежей). Любой возможный строчный тип получается путем использования конструктора ROW. При определении столбца, значения которого должны принадлежать некоторому строчному типу, используется конструкция ROW (fld1, fld2, ..., fldn), где каждый элемент fldi, определяющий поле строчного типа, задается в виде тройки **fldname, fldtype, fldoptions**. Подэлемент fldname задает имя соответствующего поля строчного типа. Подэлемент **fldtype** специфицирует тип данных этого поля. В качестве типа данных поля строчного типа можно использовать любой допустимый в SQL тип данных, включая типы коллекций, определяемые пользователями типы и другие строчные типы. Необязательный подэлемент **fldoptions** может задаваться для указания применяемого по умолчанию порядка сортировки, если соответствующий подэлемент fldtype указывает на тип символьных строк, а также должен задаваться, если fldtype указывает на ссылочный тип. *Степенью строчного типа* называется число его полей.



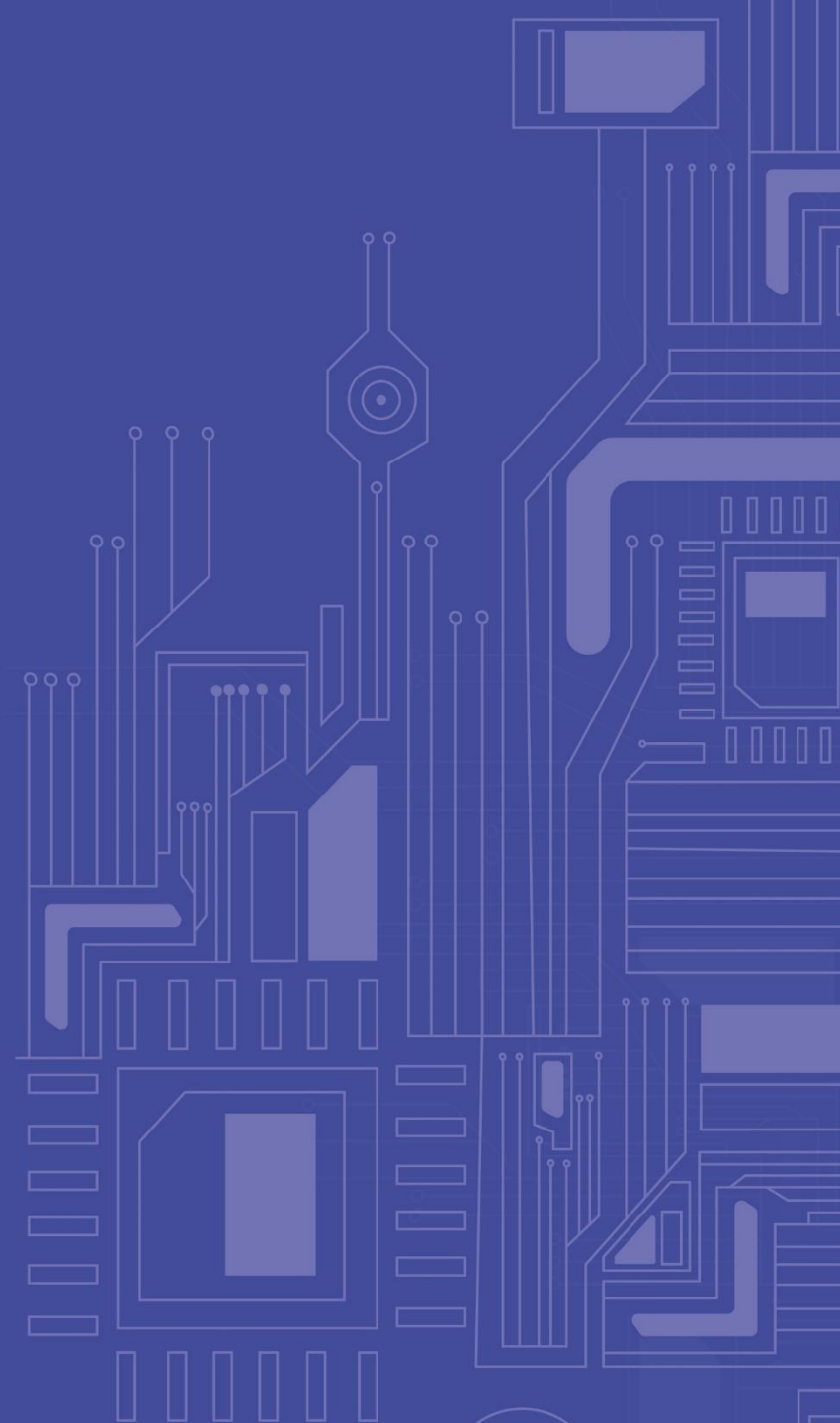
Структурные типы (Structured Types). Соответствующие возможности SQL:1999 позволяют определять долговременно хранимые, именованные типы данных, включающие один или более атрибутов любого из допустимых в SQL типа данных, в том числе другие структурные типы, типы коллекций, строчные типы и т. д. Стандарт SQL не накладывает ограничений на сложность получаемой в результате структуры данных, однако не запрещает устанавливать такие ограничения в реализации. Дополнительные механизмы определяемых пользователями методов, функций и процедур позволяют определить поведенческие аспекты структурного типа.



Индивидуальные типы (Distinct Types). Можно определить долговременно хранимый, именованный тип данных, опираясь на единственный предопределенный тип. Например, можно определить индивидуальный тип данных PRICE, опираясь на тип DECIMAL (5, 2). Тогда значения типа PRICE представляются точно так же, как значения типа DECIMAL (5, 2). Однако в SQL:1999 индивидуальный тип не наследует от своего опорного типа набор операций над значениями. Например, чтобы сложить два значения типа PRICE требуется явно сообщить системе, что с этими значениями нужно обращаться как со значениями типа DECIMAL (5, 2). Другая возможность состоит в явном определении методов, функций и процедур, связанных с данным индивидуальным типом.

Обеспечивается механизм конструирования типов (*ссылочных типов*), которые могут использоваться в качестве типов столбцов некоторого вида таблиц (типизированных таблиц). Фактически значениями ссылочного типа являются строки соответствующей типизированной таблицы. Более точно, каждой строке типизированной таблицы приписывается уникальное значение (нечто вроде первичного ключа, назначаемого системой или приложением), которое может использоваться в методах, определенных для табличного типа, для уникальной идентификации строк соответствующей таблицы. Эти уникальные значения называются *ссылочными значениями*, а их тип – ссылочным типом. Ссылочный тип может содержать только те значения, которые действительно ссылаются на экземпляры указанного типа (т. е. на строки соответствующей типизированной таблицы).

ДОМЕНЫ И ПРЕОБРАЗОВАНИЯ ТИПОВ



ПОНЯТИЕ ДОМЕНА В SQL (УСТАРЕВАЕТ)



Домен является долговременно хранимым, именованным объектом схемы базы данных. Домены можно создавать (определять), изменять (изменять определения) и ликвидировать (отменять определение). Имена доменов можно использовать при определении столбцов таблиц. Можно считать, что в SQL *определение домена* представляет собой вынесенное за пределы определения индивидуальной таблицы «родовое» определение столбца, которое можно использовать для определения различных реальных столбцов реальных базовых таблиц.

Для определения домена в SQL используется оператор CREATE DOMAIN. Общий синтаксис этого оператора следующий:

```
domain_definition ::= CREATE DOMAIN domain_name [AS] data_type  
[ default_definition ]  
[ domain_constraint_definition_list ]
```

Раздел default_definition имеет вид:

DEFAULT { literal | niladic_function | NULL }

	USER
Ниладические (служебные) функции	CURRENT_USER
	SESSION_USER
	SYSTEM_USER
	CURRENT_DATE
	CURRENT_TIME
	CURRENT_TIMESTAMP

Элемент списка **domain_constraint_definition_list** имеет вид

```
[CONSTRAINT constraint_name]  
CHECK (conditional_expression)
```

Наиболее естественным видом ограничения домена является следующий:

```
CHECK (VALUE IN (list_of_valid_values))
```

ИЗМЕНЕНИЕ И ОТМЕНА ОПРЕДЕЛЕНИЯ ДОМЕНА



ИЗМЕНЕНИЕ ОПРЕДЕЛЕНИЯ ДОМЕНА

Общий синтаксис изменения

```
domain_alteration ::=  
    ALTER DOMAIN domain_name domain_alteration_action  
domain_alteration_action ::=  
    domain_default_alteration_action  
    | domain_constraint_alteration_action
```



Изменение значения по умолчанию

```
domain_default_alteration_action ::=  
    SET default_definition  
    | DROP DEFAULT
```

Изменение ограничений

```
domain_constraint_alteration_action ::=  
    ADD domain_constraint_definition  
    | DROP CONSTRAINT constraint_name
```

ОТМЕНА ОПРЕДЕЛЕНИЯ ДОМЕНА

```
DROP DOMAIN domain_name {RESTRICT | CASCADES}
```

Если в операторе указано RESTRICT, и если соответствующий домен использован в определении некоторого столбца, в определении некоторого представления или в определении ограничения целостности, то оператор DROP DOMAIN отвергается. В противном случае определение домена ликвидируется.

Если в операторе DROP DOMAIN указано CASCADES, то оператор выполняется всегда. При этом уничтожаются все представления и ограничения целостности, в определении которых использовалось имя данного домена. Столбцы, определенные на этом домене, автоматически переопределяются следующим образом:

- считается, что каждый такой столбец теперь относится к определяющему типу уничтожаемого домена;
- если у столбца не было определено собственное значение по умолчанию, то считается, что теперь у него имеется такое значение по умолчанию, совпадающее со значением по умолчанию уничтожаемого домена;
- каждый столбец наследует все ограничения уничтожаемого домена.

ПРИМЕРЫ ОПРЕДЕЛЕНИЙ И ИЗМЕНЕНИЙ ДОМЕНОВ



```
CREATE DOMAIN EMP_NO AS INTEGER  
CHECK (VALUE BETWEEN 1 AND 10000);
```

```
CREATE DOMAIN SALARY AS NUMERIC (10, 2)  
DEFAULT 10000.00  
CHECK (VALUE BETWEEN 10000.00 AND 20000000.00)  
CONSTRAINT SAL_NOT_NULL CHECK (VALUE IS NOT NULL);
```

Немного поупражняемся с доменом SALARY. Для изменения значения заработной платы по умолчанию с 10000 на 11000 руб. нужно выполнить оператор

```
ALTER DOMAIN SALARY SET DEFAULT 11000.00;
```

Для отмены значения по умолчанию в домене SALARY следует воспользоваться оператором

```
ALTER DOMAIN SALARY DROP DEFAULT;
```

Если к определению домена SALARY требуется добавить ограничение (например, запретить значение зарплаты, равное 15000 руб.), необходимо выполнить оператор

```
ALTER DOMAIN SALARY ADD CHECK (VALUE <> 15000.00);
```

Наконец, если требуется отменить (именованное!) ограничение целостности, препятствующее наличию неопределенных значений в столбцах, которые определены на домене SALARY, то нужно выполнить оператор

```
ALTER DOMAIN SALARY DROP CONSTRAINT SAL_NOT_NULL;
```

В языке SQL обеспечивается возможность использования в различных операциях не только значений тех типов, для которых predetermined операция, но и значений типов, неявным или явным образом приводимых к требуемому типу.

В SQL поддерживается совместимость некоторых типов данных за счет *неявного преобразования* значений одного типа к значениям другого типа данных. Тип данных A приводим к типу данных B в том и только в том случае, когда в любом месте, где ожидается значение типа B, может быть использовано значение типа A.

ОСНОВНЫЕ ПРАВИЛА ПРИВОДИМОСТИ ТИПОВ

- **Типы символьных строк.** Тип CHARACTER (x) приводим к любому типу CHARACTER (y), если $y \geq x$. Типы VARCHAR (x) и CHARACTER (x) приводимы к любому типу VARCHAR (y), если $y \geq x$. Типы CHARACTER (x) и VARCHAR (x) приводимы к любому типу CLOB.
- **Типы битовых строк.** Тип BIT (x) приводим к любому типу BIT (y), если $y \geq x$. Типы BIT VARYING (x) и BIT (x) приводимы к любому типу BIT VARYING (y), если $y \geq x$.
- **Типы BLOB.** Тип BLOB (x) приводим к любому типу BLOB (y), если $y \geq x$.
- **Типы точных чисел.** Тип EN (p_1, s_1) приводим к любому типу EN (p_2, s_2), у которого $s_2 \geq s_1$ и p_2 определяется в реализации. Тип EN (p, s) приводим к любому типу приближенных чисел AN (p_1), где p_1 определяется в реализации.
- **Типы приближенных чисел.** Тип AN (p_1) приводим к любому типу AN (p_2), если $p_2 \geq p_1$.


```
CAST ({scalar-expression | NULL } AS  
      {data_type | domain_name})
```

Оператор преобразует значение заданного скалярного выражения к указанному типу или к базовому типу указанного домена. Результатом применения оператора CAST к неопределенному значению является неопределенное значение.

Примем следующие обозначения типов данных:

EN – точные числовые типы (Exact Numeric)

AN – приближительные числовые типы (Approximate Numeric)

C – типы символьных строк (Character)

FC – типы символьных строк постоянной длины (Fixed-length Character)

VC – типы символьных строк переменной длины (Variable-length Character)

B – типы битовых строк (Bit String)

FB – типы битовых строк постоянной длины (Fixed-length Bit String)

VB – типы битовых строк переменной длины (Variable-length Bit String)

D – тип Date

T – типы Time

TS – типы Timestamp

YM – типы Interval Year-Month

DT – типы Interval Day-Time

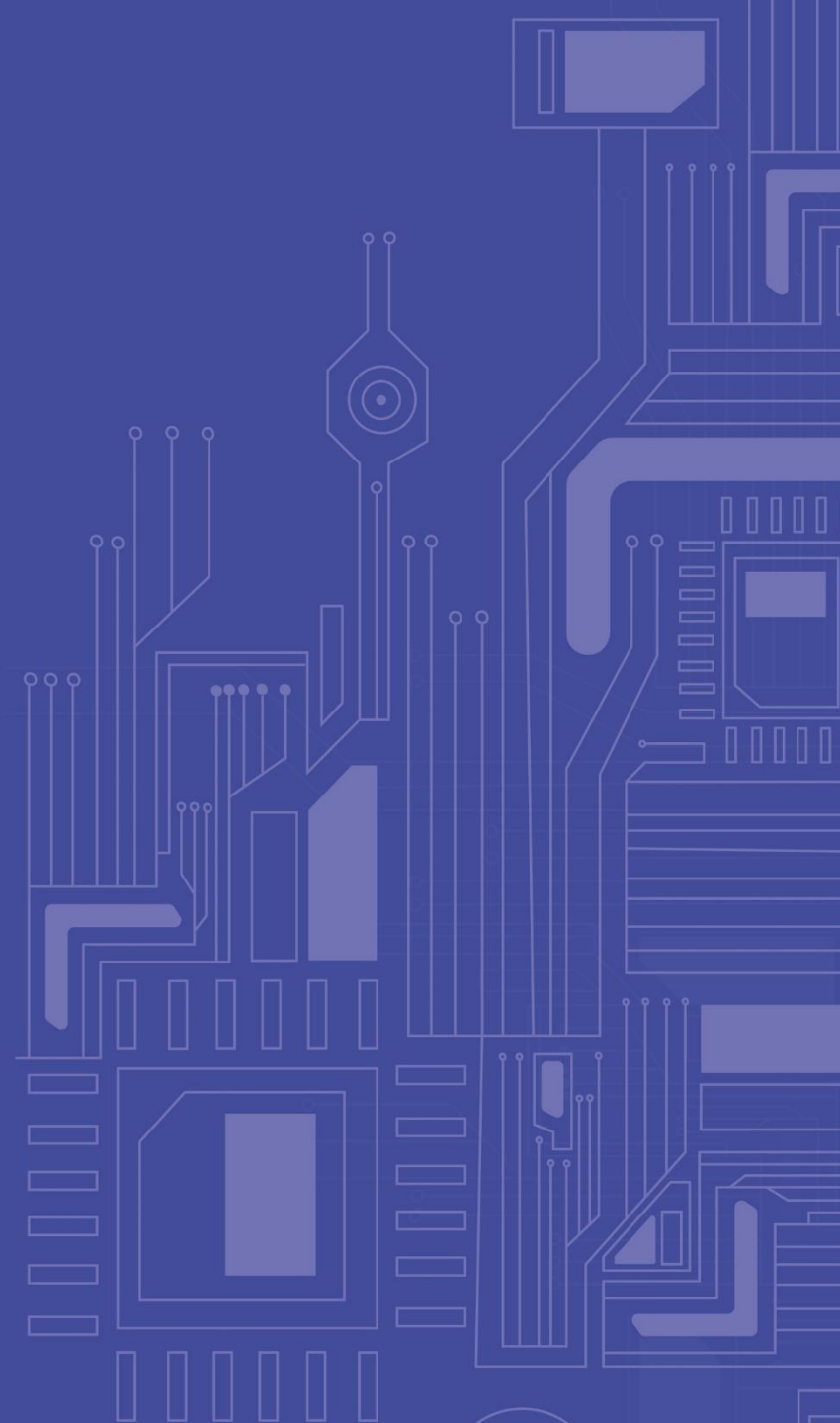
Пусть TD – это тип данных, к которому производится преобразование, а SD – тип данных операнда. Тогда допустимы следующие комбинации :

SD	TD											
		EN	AN	VC	FC	VB	FB	D	T	TS	YM	DT
	EN	Да	Да	Да	Да	Нет	Нет	Нет	Нет	Нет	?	?
	AN	Да	Да	Да	Да	Нет	Нет	Нет	Нет	Нет	Нет	Нет
	C	Да	Да	?	?	Да	Да	Да	Да	Да	Да	Да
	B	Нет	Нет	Да	Да	Да	Да	Нет	Нет	Нет	Нет	Нет
	D	Нет	Нет	Да	Да	Нет	Нет	Да	Нет	Да	Нет	Нет
	T	Нет	Нет	Да	Да	Нет	Нет	Нет	Да	Да	Нет	Нет
	TS	Нет	Нет	Да	Да	Нет	Нет	Да	Да	Да	Нет	Нет
	YM	?	Нет	Да	Да	Нет	Нет	Нет	Нет	Нет	Да	Нет
	DT	?	Нет	Да	Да	Нет	Нет	Нет	Нет	Нет	Нет	Да

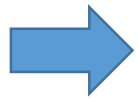
По поводу ячеек таблицы, содержащих знак вопроса, необходимо сделать несколько оговорок:

- 1. если TD – интервал и SD – тип точных чисел, то TD должен содержать единственное поле даты-времени;
- 2. если TD – тип точных чисел и SD – интервал, то SD должен содержать единственное поле даты-времени;
- 3. если SD – тип символьных строк и TD – тип символьных строк постоянной или переменной длины, то набор символов SD и TD должен быть одним и тем же.

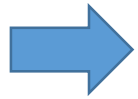
ТАБЛИЦЫ



Как и в реляционной модели данных, в модели SQL поддерживается единственная родовая структура данных, называемая в данном случае *базовой таблицей*.



Коренное отличие базовой таблицы от истинного отношения состоит в том, что тело таблицы не обязательно является множеством. Среди строк тела таблицы могут встречаться дубликаты, и в общем случае тело базовой таблицы SQL представляет собой мультимножество строк.



Порождаемые таблицы SQL, которые формируются при выполнении запросов к SQL-ориентированной базе данных, еще более отдаляют SQL от реляционной модели. В таких таблицах может отсутствовать и правильно сформированный заголовок (могут иметься одноименные столбцы).

В операторе SQL **CREATE TABLE** специфицируются не только столбцы таблицы, но и ограничения целостности, которым должны удовлетворять данные, хранящиеся в базовой таблице. Эти ограничения являются частным случаем ограничений базы данных целиком, для определения которых, а также изменения и ликвидации определений имеются специальные операторы.

Базовые (реально хранимые в базе данных) таблицы создаются (определяются) с использованием оператора **CREATE TABLE**. Для изменения определения базовой таблицы применяется оператор **ALTER TABLE**. Уничтожить хранимую таблицу (отменить ее определение) можно с помощью оператора **DROP TABLE**.

ОПРЕДЕЛЕНИЕ БАЗОВОЙ ТАБЛИЦЫ



Базовые (реально хранимые в базе данных) таблицы создаются (определяются) с использованием оператора CREATE TABLE. Для изменения определения базовой таблицы применяется оператор ALTER TABLE. Уничтожить хранимую таблицу (отменить ее определение) можно с помощью оператора DROP TABLE. Создание базовой таблицы, кроме создания соответствующих описателей, порождает новую область внешней памяти, в которой будут храниться данные, поставляемые пользователями.

```
base_table_definition ::= CREATE TABLE base_table_name (base_table_element_commalist)
base_table_element ::= column_definition | base_table_constraint_definition
```

Определение столбца

```
column_definition ::= column_name
                    { data_type | domain_name }
                    [ default_definition ]
                    [ column_constraint_definition_list ]
```

Здесь **base_table_name** задает имя новой (изначально пустой) базовой таблицы. Каждый элемент определения базовой таблицы является либо определением столбца, либо определением табличного ограничения целостности.

Значения столбца по умолчанию

```
DEFAULT { literal | niladic_function | NULL }
```

- если в определении столбца явно присутствует раздел DEFAULT, то значением столбца по умолчанию является значение, указанное в этом разделе;
- иначе, если столбец определяется на домене и в определении этого домена явно присутствует раздел DEFAULT, то значением столбца по умолчанию является значение, указанное в этом разделе;
- иначе значением по умолчанию столбца является NULL.

```
column_constraint_definition ::=  
    [ CONSTRAINT constraint_name ]  
    NOT NULL  
    | { PRIMARY KEY | UNIQUE }  
    | references_definition  
    | CHECK ( conditional_expression )
```

Ограничение NOT NULL означает, что в определяемом столбце никогда не должны содержаться неопределенные значения. Если определяемый столбец имеет имя C, то это ограничение эквивалентно следующему табличному ограничению: CHECK (C IS NOT NULL).

В определение столбца может входить одно из ограничений уникальности: ограничение первичного ключа (PRIMARY KEY) или ограничение возможного ключа (UNIQUE). Ограничение PRIMARY KEY, в дополнение к требованию NOT NULL значений самого столбца, влечет за собой ограничение NOT NULL для определяемого столбца. Эти ограничения столбца эквивалентны следующим табличным ограничениям: PRIMARY KEY (C) и UNIQUE (C).

```
references_definition ::=  
    REFERENCES base_table_name [ (column_comma_list) ]  
    [ MATCH { SIMPLE | FULL | PARTIAL } ]  
    [ ON DELETE referential_action ]  
    [ ON UPDATE referential_action ]
```

Ограничение **references_definition** означает объявление определяемого столбца внешним ключом таблицы. Ограничение эквивалентно следующему табличному ограничению: FOREIGN KEY (C) references_definition.

Проверочное ограничение CHECK (conditional_expression) приводит к тому, что в данном столбце могут находиться только те значения, для которых вычисление conditional_expression не приводит к результату false. В условном выражении проверочного ограничения столбца разрешается использовать имя только определяемого столбца.

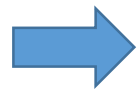
ТАБЛИЧНЫЕ ОГРАНИЧЕНИЯ (1/2)



```
base_table_constraint_definition ::=  
  [ CONSTRAINT constraint_name ]  
  { PRIMARY KEY | UNIQUE } ( column_comma_list )  
  | FOREIGN KEY ( column_comma_list )  
    references_definition  
  | CHECK ( conditional_expression )
```

Имеется три разновидности табличных ограничений: ограничение первичного или возможного ключа (PRIMARY KEY или UNIQUE), ограничение внешнего ключа (FOREIGN KEY) и проверочное ограничение (CHECK). Любому ограничению может явным образом назначаться имя, если перед определением ограничения поместить конструкцию CONSTRAINT constraint_name.

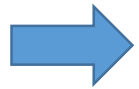
Ограничение первичного или возможного ключа



Табличное ограничение первичного или возможного ключа { PRIMARY_KEY | UNIQUE } (column_comma_list) означает требование уникальности составных значений указанной группы столбцов. Ограничение PRIMARY KEY, в дополнение к этому, влечет ограничение NOT NULL для всех столбцов, упоминаемых в определении ограничения. В определении таблицы допускается произвольное число определений возможного ключа (для разных комбинаций столбцов), но не более одного определения первичного ключа.

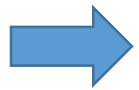
В языке SQL действительно допускается определение таблиц, у которых отсутствуют возможные ключи. Эта особенность языка, среди прочего, очевидным образом противоречит базовым требованиям реляционной модели данных.

Проверочное табличное ограничение



Определение табличного ограничения вида CHECK (conditional_expression) приводит к тому, что указанное условное выражение будет вычисляться при каждой попытке обновления соответствующей таблицы. Таблица находится в соответствии с данным проверочным табличным ограничением, если для всех строк таблицы результатом вычисления соответствующего условного выражения не является *false*.

Табличное ограничение внешнего ключа



Табличное ограничение FOREIGN KEY (column_commalist) references_definition означает объявление внешним ключом группы столбцов, имена которых перечислены в списке column_commalist.

```
references_definition ::=  
    REFERENCES base_table_name [ (column_commalist) ]  
        [ MATCH { SIMPLE | FULL | PARTIAL } ]  
        [ ON DELETE referential_action ]  
        [ ON UPDATE referential_action ]
```

Если определение ссылок включает список столбцов (column_commalist), то этот список должен совпадать со списком имен столбцов, использованных в некотором определении первичного или возможного ключа (PRIMARY_KEY или UNIQUE) в определении таблицы base_table_name (T). Если в определении ссылок список столбцов явно не задан, то считается, что он совпадает со списком столбцов, использованных в определении первичного ключа (PRIMARY_KEY) таблицы T.

ТАБЛИЧНОЕ ОГРАНИЧЕНИЕ ВНЕШНЕГО КЛЮЧА (1/2)



Разновидности способов сопоставления значений внешнего и возможного ключей

Пусть определяемая таблица имеет имя *S*, а базовая таблица *base_table_name* имеет имя *T*.

СМЫСЛ НЕОБЯЗАТЕЛЬНОГО РАЗДЕЛА ОПРЕДЕЛЕНИЯ ВНЕШНЕГО КЛЮЧА MATCH { SIMPLE | FULL | PARTIAL }.

Ограничение внешнего ключа удовлетворяется в том и только в том случае, когда для каждой строки таблицы *S* выполняется одно из следующих условий:

Раздел отсутствует или имеет вид MATCH SIMPLE

- какой-либо столбец, входящий в состав внешнего ключа, содержит NULL;
- таблица *T* содержит в точности одну строку, такую, что значение внешнего ключа в данной строке таблицы *S* совпадает со значением соответствующего возможного ключа в этой строке таблицы *T*.

Раздел MATCH присутствует и имеет вид MATCH PARTIAL

- каждый столбец, входящий в состав внешнего ключа, содержит NULL;
- таблица *T* содержит по крайней мере одну такую строку, что для каждого столбца данной строки таблицы *S*, значение которого отлично от NULL, его значение совпадает со значением соответствующего столбца возможного ключа в этой строке таблицы *T*.

ТАБЛИЧНОЕ ОГРАНИЧЕНИЕ ВНЕШНЕГО КЛЮЧА (2/2)



Раздел **MATCH** имеет вид **MATCH FULL**

- каждый столбец, входящий в состав внешнего ключа, содержит NULL;
- ни один столбец, входящий в состав внешнего ключа, не содержит NULL, и таблица T содержит в точности одну строку, такую, что значение внешнего ключа в данной строке таблицы S совпадает со значением соответствующего возможного ключа в этой строке таблицы T.

Только при наличии спецификации **MATCH FULL** ссылочное ограничение соответствует требованиям реляционной модели. Тем не менее, в определении ограничения внешнего ключа базовых таблиц в SQL по умолчанию предполагается наличие спецификации **MATCH SIMPLE**.

Поддержка ссылочной целостности и ссылочные действия

В связи с определением ограничения внешнего ключа осталось рассмотреть еще два необязательных раздела – **ON DELETE** `referential_action` и **ON UPDATE** `referential_action`.

```
referential_action ::=  
    { NO ACTION | RESTRICT | CASCADE  
      | SET DEFAULT | SET NULL }
```

Чтобы объяснить, в каких случаях и каким образом выполняются эти действия, требуется сначала определить понятие ссылающейся строки (`referencing row`).

В определении ограничения внешнего ключа **отсутствует раздел MATCH** или **присутствуют** спецификации **MATCH SIMPLE** либо **MATCH FULL**

Тогда для данной строки **t** таблицы **T** строкой таблицы **S**, ссылающейся на строку **t**, называется каждая строка таблицы **S**, значение внешнего ключа которой совпадает со значением соответствующего возможного ключа строки **t**.

В определении ограничения внешнего ключа **присутствует** спецификация **MATCH PARTIAL**

Тогда для данной строки **t** таблицы **T** строкой таблицы **S**, ссылающейся на строку **t**, называется каждая строка таблицы **S**, отличные от NULL значения столбцов внешнего ключа которой совпадают со значениями соответствующих столбцов соответствующего возможного ключа строки **t**.



В случае **MATCH PARTIAL** строка таблицы **S** называется ссылающейся исключительно на строку **t** таблицы **T**, если эта строка таблицы **S** является ссылающейся на строку **t** и не является ссылающейся на какую-либо другую строку таблицы **T**.

ССЫЛОЧНЫЕ ДЕЙСТВИЯ (ON DELETE)

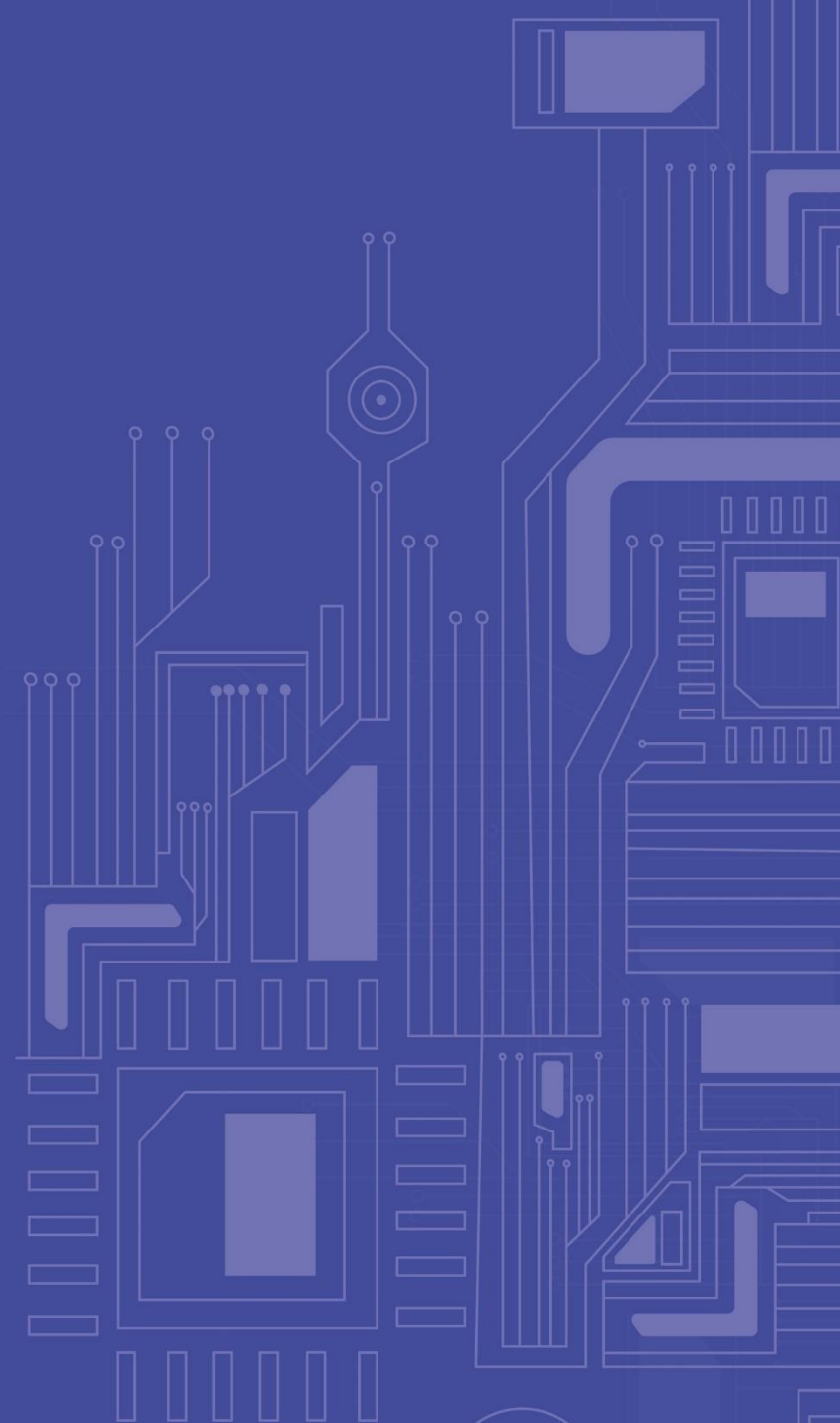


Пусть определение ограничения внешнего ключа содержит раздел **ON DELETE** `referential_action`. Предположим, что предпринимается попытка удалить строку `t` из таблицы `T`. Тогда:

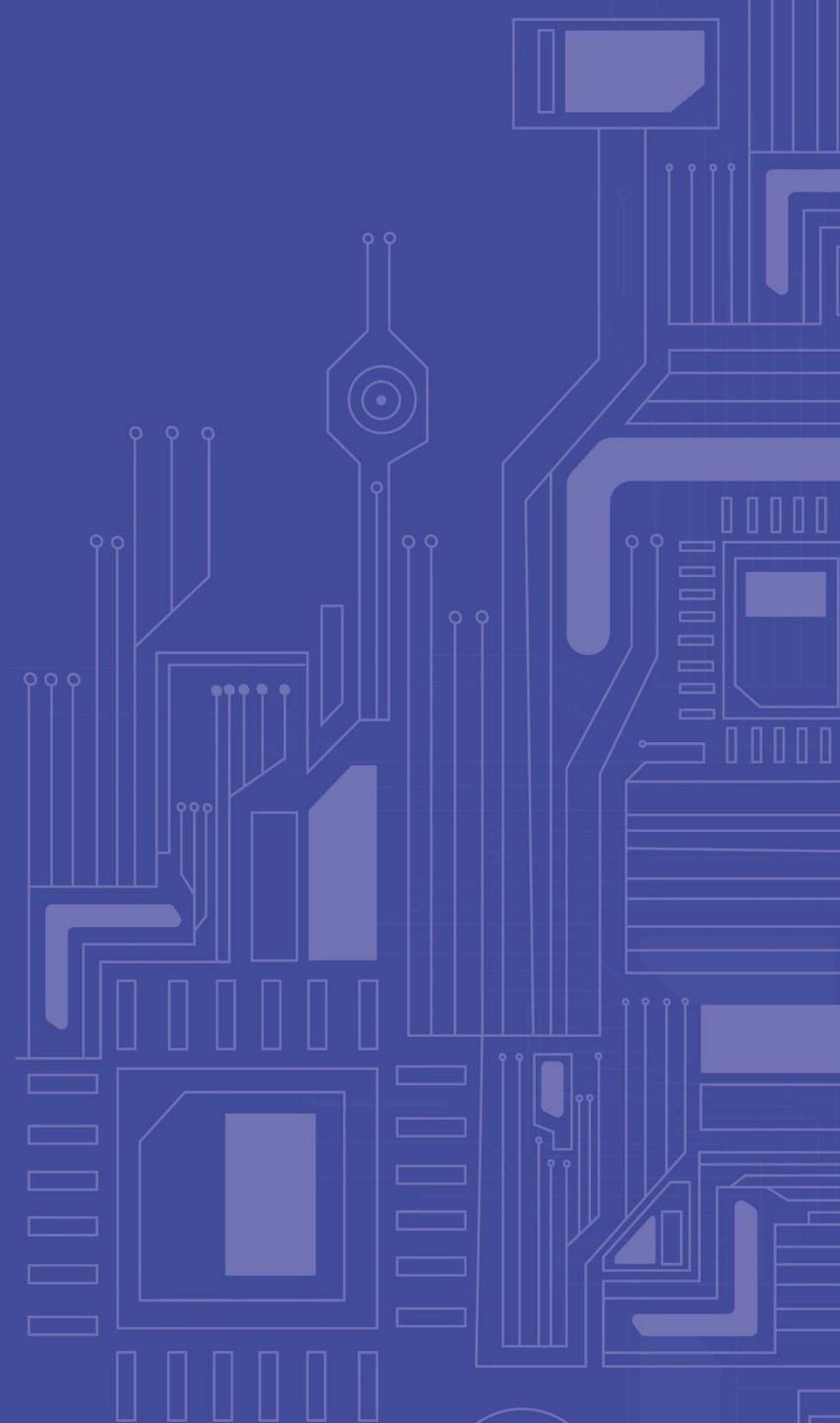
- если в качестве требуемого ссылочного действия указано `NO ACTION` или `RESTRICT`, то операция удаления отвергается, если ее выполнение вызвало бы нарушение ограничения внешнего ключа;
- если в качестве требуемого ссылочного действия указано `CASCADE`, то строка `t` удаляется, и если в определении ограничения внешнего ключа отсутствует раздел `MATCH` или присутствуют спецификации `MATCH SIMPLE` или `MATCH FULL`, то удаляются все строки, ссылающиеся на `t`. Если же в определении ограничения внешнего ключа присутствует спецификация `MATCH PARTIAL`, то удаляются только те строки, которые ссылаются исключительно на строку `t`;
- если в качестве требуемого ссылочного действия указано `SET DEFAULT`, то строка `t` удаляется, и во всех столбцах, которые входят в состав внешнего ключа, всех строк, ссылающихся на строку `t`, проставляется заданное при их определении значение по умолчанию. Если в определении внешнего ключа содержится спецификация `MATCH PARTIAL`, то подобному воздействию подвергаются только те строки таблицы `S`, которые ссылаются исключительно на строку `t`;
- если в качестве требуемого ссылочного действия указано `SET NULL`, то строка `t` удаляется, и во всех столбцах, которые входят в состав внешнего ключа, всех строк, ссылающихся на строку `t`, проставляется `NULL`. Если в определении внешнего ключа содержится спецификация `MATCH PARTIAL`, то подобному воздействию подвергаются только те строки таблицы `S`, которые ссылаются исключительно на строку `t`.

Пусть определение ограничения внешнего ключа содержит раздел **ON UPDATE** `referential_action`. Предположим, что предпринимается попытка обновить столбцы соответствующего возможного ключа в строке `t` из таблицы `T`. Тогда:

- если в качестве требуемого ссылочного действия указано `NO ACTION` или `RESTRICT`, то операция обновления отвергается, если ее выполнение вызвало бы нарушение ограничения внешнего ключа;
- если в качестве требуемого ссылочного действия указано `CASCADE`, то строка `t` обновляется, и если в определении ограничения внешнего ключа отсутствует раздел `MATCH` или присутствуют спецификации `MATCH SIMPLE` или `MATCH FULL`, то соответствующим образом обновляются все строки, ссылающиеся на `t` (в них должным образом изменяются значения столбцов, входящих в состав внешнего ключа). Если же в определении ограничения внешнего ключа присутствует спецификация `MATCH PARTIAL`, то обновляются только те строки, которые ссылаются исключительно на строку `t`;
- если в качестве требуемого ссылочного действия указано `SET DEFAULT`, то строка `t` обновляется, и во всех столбцах, которые входят в состав внешнего ключа и соответствуют изменяемым столбцам таблицы `T`, всех строк, ссылающихся на строку `t`, проставляется заданное при их определении значение по умолчанию. Если в определении внешнего ключа содержится спецификация `MATCH PARTIAL`, то подобному воздействию подвергаются только те строки таблицы `S`, которые ссылаются исключительно на строку `t`, причем в них изменяются значения только тех столбцов, которые не содержали `NULL`;
- если в качестве требуемого ссылочного действия указано `SET NULL`, то строка `t` обновляется, и во всех столбцах, которые входят в состав внешнего ключа и соответствуют изменяемым столбцам таблицы `T`, всех строк, ссылающихся на строку `t`, проставляется `NULL`. Если в определении внешнего ключа содержится спецификация `MATCH PARTIAL`, то подобному воздействию подвергаются только те строки таблицы `S`, которые ссылаются исключительно на строку `t`.



СЕМИНАР



ПРИМЕРЫ ОПРЕДЕЛЕНИЯ БАЗОВЫХ ТАБЛИЦ (1/6)



EMP:

EMP_NO : EMP_NO
EMP_NAME : VARCHAR
EMP_BDATE : DATE
EMP_SAL : SALARY
DEPT_NO : DEPT_NO
PRO_NO : PRO_NO

```
(1) CREATE TABLE EMP (  
(2) EMP_NO EMP_NO PRIMARY KEY,  
(3) EMP_NAME VARCHAR(20) DEFAULT 'Incognito' NOT NULL,  
(4) EMP_BDATE DATE DEFAULT NULL CHECK (  
VALUE >= DATE '1917-10-24'),  
(5) EMP_SAL SALARY,  
(6) DEPT_NO DEPT_NO DEFAULT NULL REFERENCES  
DEPT ON DELETE SET NULL,  
(7) PRO_NO PRO_NO DEFAULT NULL,  
(8) FOREIGN KEY PRO_NO REFERENCES PRO (PRO_NO)  
ON DELETE SET NULL,  
(9) CONSTRAINT PRO_EMP_NO CHECK  
((SELECT COUNT (*) FROM EMP E  
WHERE E.PRO_NO = PRO_NO) <= 50));
```

Столбцы EMP_NO, EMP_SAL, DEPT_NO, PRO_NO, DEPT_TOTAL_SAL, DEPT_MNG и PRO_MNG определяются на ранее определенных доменах (определения доменов EMP_NO и SALARY приведены в лекции). Первичными ключами отношений EMP, DEPT и проектов PRO являются столбцы EMP_NO, DEPT_NO и PRO_NO соответственно. В таблице EMP столбцы DEPT_NO и PRO_NO являются внешними ключами, указывающими на отдел, в котором работает служащий, и на выполняемый им проект соответственно. В таблице DEPT внешним ключом является столбец DEPT_NO, указывающий на служащего, являющегося руководителем соответствующего отдела, а в таблице PRO внешним ключом является столбец PRO_MNG, указывающий на служащего, являющегося менеджером соответствующего проекта.

ПРИМЕРЫ ОПРЕДЕЛЕНИЯ БАЗОВЫХ ТАБЛИЦ (2/6)



```
(1) CREATE TABLE EMP (  
(2) EMP_NO EMP_NO PRIMARY KEY,  
(3) EMP_NAME VARCHAR(20) DEFAULT 'Incognito' NOT NULL,  
(4) EMP_BDATE DATE DEFAULT NULL CHECK (  
    VALUE >= DATE '1917-10-24'),  
(5) EMP_SAL SALARY,  
(6) DEPT_NO DEPT_NO DEFAULT NULL REFERENCES  
    DEPT ON DELETE SET NULL,  
(7) PRO_NO PRO_NO DEFAULT NULL,  
(8) FOREIGN KEY PRO_NO REFERENCES PRO (PRO_NO)  
    ON DELETE SET NULL,  
(9) CONSTRAINT PRO_EMP_NO CHECK  
    ((SELECT COUNT (*) FROM EMP E  
        WHERE E.PRO_NO = PRO_NO) <= 50));
```

В части (1) указывается, что создается таблица с именем EMP. В части (2) определяется столбец EMP_NO на домене EMP_NO. У этого столбца не определено значение по умолчанию, и он объявлен первичным ключом таблицы (это ограничение целостности добавляется через AND к ограничениям, унаследованным столбцом от определения домена). Помимо прочего, это означает неявное указание запрета для данного столбца неопределенных значений.

В части (3) определен столбец EMP_NAME на базовом типе данных символьных строк переменной длины с максимальной длиной 20. Для столбца указано значение по умолчанию – строка 'Incognito', и в качестве ограничения целостности запрещены неопределенные значения. В части (4) определяется столбец EMP_BDATE (дата рождения служащего). Он имеет тип данных DATE, значением по умолчанию является NULL (даты рождения некоторых служащих неизвестны). Кроме того, ограничение столбца запрещает принимать на работу лиц, о которых известно, что они родились до Октябрьского переворота. В части (5) определен столбец EMP_SAL на домене SALARY. Значение по умолчанию и ограничения целостности наследуются из определения домена.

ПРИМЕРЫ ОПРЕДЕЛЕНИЯ БАЗОВЫХ ТАБЛИЦ (3/6)



```
(1) CREATE TABLE EMP (  
(2) EMP_NO EMP_NO PRIMARY KEY,  
(3) EMP_NAME VARCHAR(20) DEFAULT 'Incognito' NOT NULL,  
(4) EMP_BDATE DATE DEFAULT NULL CHECK (  
VALUE >= DATE '1917-10-24'),  
(5) EMP_SAL SALARY,  
(6) DEPT_NO DEPT_NO DEFAULT NULL REFERENCES  
DEPT ON DELETE SET NULL,  
(7) PRO_NO PRO_NO DEFAULT NULL,  
(8) FOREIGN KEY PRO_NO REFERENCES PRO (PRO_NO)  
ON DELETE SET NULL,  
(9) CONSTRAINT PRO_EMP_NO CHECK  
((SELECT COUNT (*) FROM EMP E  
WHERE E.PRO_NO = PRO_NO) <= 50));
```

В части (6) столбец DEPT_NO определяется на одноименном домене (для наших целей его определение несущественно), но явно объявляется, что значением по умолчанию этого столбца будет NULL (некоторые служащие не приписаны ни к какому отделу). Кроме того, добавляется ограничение внешнего ключа: столбец DEPT_NO ссылается на первичный ключ таблицы DEPT. Определено ссылочное действие: при удалении строки из таблицы DEPT во всех строках таблицы EMP, ссылавшихся на эту строку, столбцу DEPT_NO должно быть присвоено неопределенное значение.

В части (7) определяется столбец PRO_NO. Его определение аналогично определению столбца DEPT_NO, но ограничение внешнего ключа вынесено в часть (8), где оно определяется в полной форме как табличное ограничение. Наконец, в части (9) определяется табличное проверочное ограничение с именем PRO_EMP_NO, которое требует, чтобы ни в одном проекте не участвовало больше 50 служащих.

ПРИМЕРЫ ОПРЕДЕЛЕНИЯ БАЗОВЫХ ТАБЛИЦ

(4/6)



DEPT:

DEPT_NO : DEPT_NO
DEPT_NAME : VARCHAR
DEPT_EMP_NO : INTEGER
DEPT_TOTAL_SAL : SALARY
DEPT_MNG : EMP_NO

В части (3) столбец DEPT_EMP_NO (число служащих в отделе) определен на базовом типе INTEGER без значения по умолчанию, с запретом неопределенного значения и с проверочным ограничением, устанавливающим допустимый диапазон значений числа служащих в отделе. Еще одно проверочное ограничение этого столбца – (7) – вынесено на уровень определения табличного ограничения. Это ограничение устанавливает, что в каждой строке таблицы DEPT значение столбца DEPT_EMP_NO должно равняться общему числу строк таблицы EMP, в которых значение столбца DEPT_NO равно значению одноименного столбца данной строки таблицы DEPT.

```
(1) CREATE TABLE DEPT (  
(2) DEPT_NO DEPT_NO PRIMARY KEY,  
(3) DEPT_EMP_NO INTEGER NO NULL CHECK (  
    VALUE BETWEEN 1 AND 100),  
(4) DEPT_NAME VARCHAR(200) DEFAULT 'Nameless' NOT NULL,  
(5) DEPT_TOTAL_SAL SALARY DEFAULT 1000000.00  
    NO NULL CHECK (VALUE >= 100000.00),  
(6) DEPT_MNG EMP_NO DEFAULT NULL  
    REFERENCES EMP ON DELETE SET NULL  
    CHECK (IF (VALUE IS NOT NULL) THEN  
        ((SELECT COUNT(*) FROM DEPT  
            WHERE DEPT.DEPT_MNG = VALUE) = 1),  
(7) CHECK (DEPT_EMP_NO =  
    (SELECT COUNT(*) FROM EMP  
        WHERE DEPT_NO = EMP.DEPT_NO)),  
(8) CHECK (DEPT_TOTAL_SAL >=  
    (SELECT SUM(EMP_SAL) FROM EMP  
        WHERE DEPT_NO = EMP.DEPT_NO)));
```

В части (5) для определения столбца DEPT_TOTAL_SAL (объем фонда заработной платы отдела) используется домен SALARY. Но при этом явно установлено значение столбца по умолчанию (отличное от значения по умолчанию домена), запрещено наличие неопределенных значений и введено дополнительное проверочное ограничение, определяющее нижний порог объема фонда заработной платы отдела. Еще одно проверочное ограничение – (8) – вынесено на уровень определения табличного ограничения. Это ограничение устанавливает, что в каждой строке таблицы DEPT значение столбца DEPT_TOTAL_SAL должно быть не меньше суммы значений столбца EMP_SAL во всех строках таблицы EMP, в которых значение столбца DEPT_NO равно значению одноименного столбца данной строки таблицы DEPT.

ПРИМЕРЫ ОПРЕДЕЛЕНИЯ БАЗОВЫХ ТАБЛИЦ

(5/6)



```
(1) CREATE TABLE DEPT (  
(2) DEPT_NO DEPT_NO PRIMARY KEY,  
(3) DEPT_EMP_NO INTEGER NO NULL CHECK (  
    VALUE BETWEEN 1 AND 100),  
(4) DEPT_NAME VARCHAR(200) DEFAULT 'Nameless' NOT NULL,  
(5) DEPT_TOTAL_SAL SALARY DEFAULT 1000000.00  
    NO NULL CHECK (VALUE >= 100000.00),  
(6) DEPT_MNG EMP_NO DEFAULT NULL  
    REFERENCES EMP ON DELETE SET NULL  
    CHECK (IF (VALUE IS NOT NULL) THEN  
        ((SELECT COUNT(*) FROM DEPT  
            WHERE DEPT.DEPT_MNG = VALUE) = 1),  
(7) CHECK (DEPT_EMP_NO =  
    (SELECT COUNT(*) FROM EMP  
        WHERE DEPT_NO = EMP.DEPT_NO)),  
(8) CHECK (DEPT_TOTAL_SAL >=  
    (SELECT SUM(EMP_SAL) FROM EMP  
        WHERE DEPT_NO = EMP.DEPT_NO)));
```

Определение столбца DEPT_MNG – часть (6). Этот столбец объявляется внешним ключом таблицы DEPT. Но мы хотим сказать больше. У отдела могут временно отсутствовать руководители, поэтому в столбце допускаются неопределенные значения. Но если у отдела имеется руководитель, то он должен являться руководителем только этого отдела. На первый взгляд можно было бы воспользоваться ограничением столбца UNIQUE. Но такое ограничение допускало бы наличие неопределенного столбца DEPT_MNG только в одной строке таблицы DEPT, а мы хотим допустить отсутствие руководителя у нескольких отделов. Поэтому потребовалось ввести более громоздкое проверочное ограничение столбца.

Ограничение (9) в первом определении и ограничения (7) и (8) во втором определении внешне похожи, но сильно отличаются по своей сути. Ограничения (7) и (8) связаны с агрегатной семантикой столбцов DEPT_EMP_NO и DEPT_TOTAL_SAL таблицы DEPT. Отмена ограничений изменила бы смысл этих столбцов. Ограничение (9) является текущим административным ограничением. Если руководство предприятия примет решение разрешить использовать в одном проекте более 50 служащих, ограничение можно отменить без изменения смысла столбцов таблицы EMP. Имея это в виду, мы ввели явное имя ограничения (9), чтобы при необходимости имелась простая возможность отменить это ограничение с помощью оператора ALTER TABLE.

ПРИМЕРЫ ОПРЕДЕЛЕНИЯ БАЗОВЫХ ТАБЛИЦ

(6/6)



PRO:

PRO_NO : PRO_NO
PRO_TITLE : VARCHAR
PRO_SDATE : DATE
PRO_DURAT : INTERVAL
PRO_MNG : EMP_NO
PRO_DESC : CLOB

```
(1) CREATE TABLE PRO (  
(2) PRO_NO PRO_NO PRIMARY KEY,  
(3) PRO_TITLE VARCHAR(200)DEFAULT 'No title' NOT NULL,  
(4) PRO_SDATE DATE DEFAULT CURRENT_DATE NOT NULL,  
(5) PRO_DURAT INTERVAL YEAR DEFAUL INTERVAL '1'  
YEAR NOT NULL,  
(6) PRO_MNG EMP_NO UNIQUE NOT NULL  
REFERENCES EMP ON DELETE NO ACTION,  
(7) PRO_DESC CLOB(10M));
```

Столбец PRO_SDATE содержит дату начала проекта, а столбец PRO_DURAT – продолжительность проекта в годах. В этом определении имеет смысл прокомментировать часть (6). Мы считаем, что если отдел, по крайней мере временно, может существовать без руководителя, то у проекта всегда должен быть менеджер. Поэтому определение столбца PRO_MNG является гораздо более строгим, чем определение столбца DEPT_MNG в таблице DEPT. Сочетание ограничений UNIQUE и NOT NULL при отсутствии значений по умолчанию приводит к абсолютной уникальности значений столбца PRO_MNG. Другими словами, этот столбец обладает всеми характеристиками первичного ключа, хотя объявлен только как возможный ключ. Кроме того, он объявлен как внешний ключ с действием при удалении строки таблицы EMP с соответствующим значением первичного ключа NO ACTION, запрещающим такие удаления. В совокупности это гарантирует, что у любого проекта будет существовать менеджер, являющийся служащим предприятия. В части (5) столбец PRO_DESC (описание проекта) определен как большой символьный объект с максимальным размером 10 Мбайт.

Как получить список всех таблиц (включая служебные) из разных БД:

-- H2, HSQLDB, MySQL, PostgreSQL, SQL Server

```
SELECT table_schema, table_name  
FROM information_schema.tables
```

-- DB2

```
SELECT tabschema, tabname  
FROM syscat.tables
```

-- Oracle

```
SELECT owner, table_name  
FROM all_tables
```

-- SQLite

```
SELECT name  
FROM sqlite_master
```

-- Teradata

```
SELECT databasename, tablename  
FROM dbc.tables
```

ПРИМЕР СОЗДАНИЯ ТАБЛИЦ В MS SQL



Создать *таблицу* для хранения данных о товарах, поступающих в продажу в некоторой торговой фирме. Необходимо учесть такие сведения, как название и тип товара, его цена, сорт и город, где товар производится.

```
CREATE TABLE Товар
(Название      VARCHAR(50) NOT NULL,
  Цена          MONEY NOT NULL,
  Тип           VARCHAR(50) NOT NULL,
  Сорт          VARCHAR(50),
  ГородТовара  VARCHAR(50))
```

Создать таблицу для сохранения сведений о постоянных клиентах с указанием названий города и фирмы, фамилии, имени и отчества клиента, номера его телефона.

```
CREATE TABLE Клиент
(Фирма        VARCHAR(50) NOT NULL,
  Фамилия      VARCHAR(50) NOT NULL,
  Имя          VARCHAR(50) NOT NULL,
  Отчество     VARCHAR(50),
  ГородКлиента VARCHAR(50),
  Телефон      CHAR(10) NOT NULL)
```

Составить *список* сведений о всех клиентах.

```
SELECT * FROM Клиент
```

Составить список всех фирм.

```
SELECT ALL Клиент.Фирма  
FROM Клиент
```

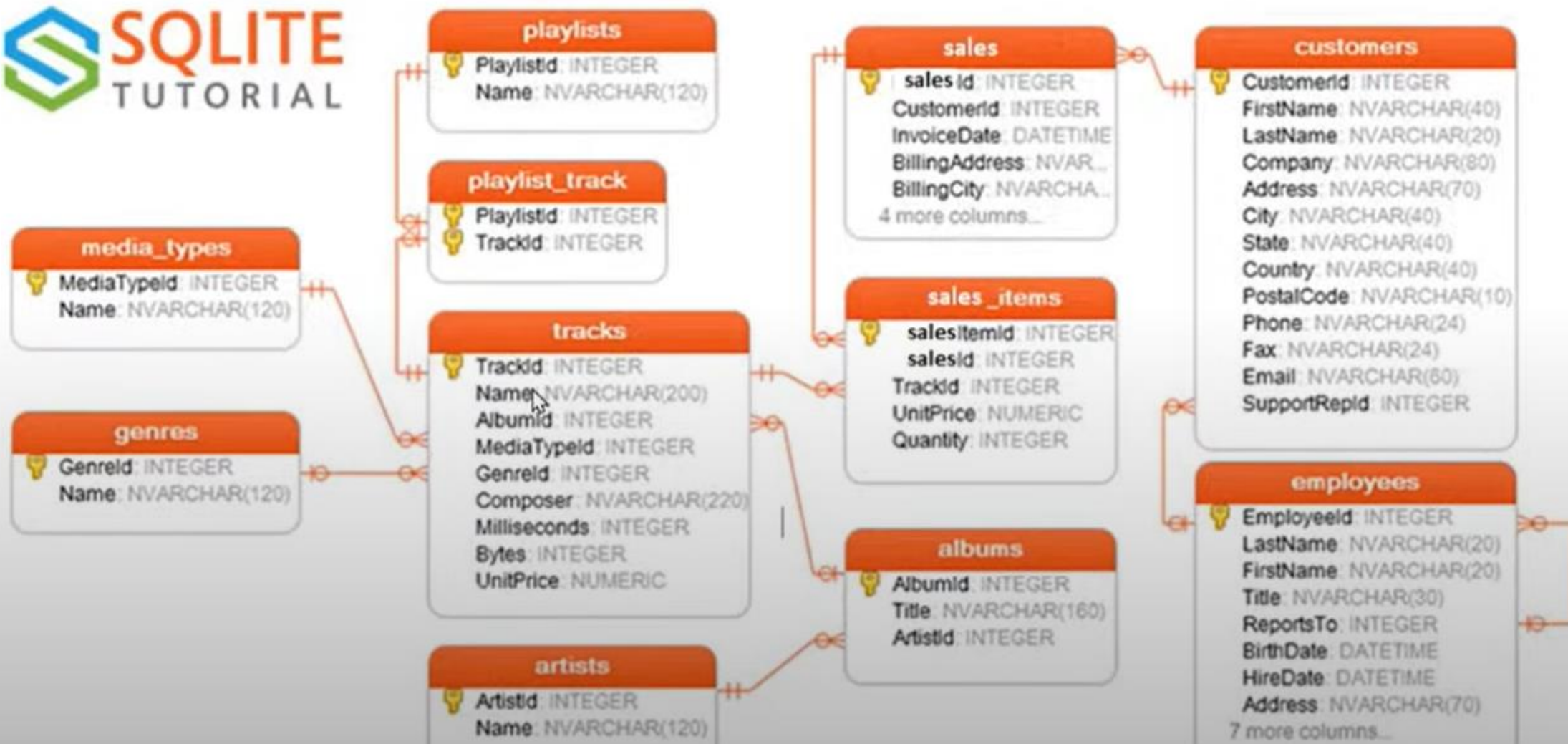
Или (что эквивалентно)

```
SELECT Клиент.Фирма  
FROM Клиент
```

Составить список всех уникальных фирм.

```
SELECT DISTINCT Клиент.Фирма  
FROM Клиент
```

СТРУКТУРА ДАННЫХ УЧЕБНОЙ БАЗЫ



ОБЗОР КОМАНД SQL ДЛЯ SQLITE3



```
1
2  DML:--Data Manipulation Language Язык определения данных
3      SELECT DCL
4      INSERT
5      UPDATE
6      DELETE
7      MERGE
8  DDL:--Data Defenition Language Язык манипулирования данными
9      CREATE
10     ALTER
11     DROP
12
13  DCL:--Data Control Language (DCL) язык управления данными
14     GRANT
15     REVOKE
16     DENY
17  TCL  --transaction Control Language язык управления транзакциями
18     BEGIN
19     COMMIT
20     ROLLBACK
```

ТИПЫ ЗАПРОСОВ (1/2)



```
1
2
3 SELECT *
4 from tracks
5 where UnitPrice>1 and UnitPrice<2 and name <> 'Torn'
```

Конкатенация

```
1 select LastName,FirstName,LastName || ', ' || FirstName as fio
2 from employees
```

Обработка NULL

```
1 select    FirstName || ', ' || LastName as FIO,
2           ifnull(Address,'') || ', ' || ifnull(City,'') || ', ' || ifnull(State,'') || ', ' || ifnull(Country,'') as full_adress
3           ,FAX
4
5
6 from customers
```

Поиск по слову

```
1 -- 1.09 Отобразить все песни в которых названии содержится слово night.
2
3 select *
4 from tracks
5 where name like '%night%'
```

Комментарии

```
1 --1.05 Отобразить все музыкальные треки с ценой меньше 1
2 /*dfg
3 dfg
4 fbvbrnagd
5 */
```

ТИПЫ ЗАПРОСОВ (2/2)



Ограничение на значение

```
3 select *
4 from tracks
5 where TrackId in (1,2,3,4,5)
```

I

```
3 select *
4 from tracks
5 where Composer='AC/DC' and unitPrice < 0.5 or name like 'A%|
```

Помним о приоритетах, у AND он выше, чем у OR

Работа с датами

```
4 select * I
5 from sales
6 where SalesDate >= date('2010-01-01') and SalesDate < date('2011-01-01')
```

Группировка с выбором верхних двух

```
SELECT date(salesDate, 'start of year') as yyyy , count(*)
FROM sales
group by date(salesDate, 'start of year')
having count(*)>8
order by yyyy limit 2|
```

Округление дат

```
select SalesDate
, date(SalesDate)
, date(SalesDate, 'start of month' )
, date(SalesDate, 'start of year' )
, time(salesDate)
, SalesDate - date(SalesDate, 'start of month' )
, strftime('%d', salesDate) as DD
, strftime('%m', salesDate) as MM
, strftime('%Y', salesDate) as YYYY
from sales
where --SalesDate >= date('2010-01-01') and SalesDate < date('2011-01-01')
date(SalesDate, 'start of year' ) = date('2010-01-01')
```

I

Группировка с отображением количества строк и среднего по каждой группе

```
7 select country, count(*), avg(age)
8 from customers
9 group by country
```


1. Покажите фамилию и имя клиентов из города Прага ?
2. Покажите фамилию и имя клиентов у которых имя начинается букву М ? Содержит символ "ch"?
3. Покажите название и размер музыкальных треков в Мегабайтах ?
4. Покажите фамилию и имя сотрудников компании нанятых в 2002 году из города Калгари ?
5. Покажите фамилию и имя сотрудников компании нанятых в возрасте 40 лет и выше?
6. Покажите покупателей-американцев без факса ?
7. Покажите канадские города в которые сделаны продажи в августе и сентябре месяце?
8. Покажите почтовые адреса клиентов из домена gmail.com ?
9. Покажите сотрудников которые работают в компании уже 18 лет и более ?
10. Покажите в алфавитном порядке все должности в компании ?
11. Покажите в алфавитном порядке Фамилию, Имя и год рождения покупателей ?
12. Сколько секунд длится самая короткая песня ?
13. Покажите название и длительность в секундах самой короткой песни ?
14. Покажите средний возраст клиента для каждой страны ?
15. Покажите Фамилии работников нанятых в октябре?
16. Покажите фамилию самого старого по стажу сотрудника в компании ?